

SAPIENZA
UNIVERSITÀ DI ROMA

"Sapienza" University of Rome
Faculty of Information Engineering,
Informatics and Statistics
Department of Computer Science

FDS

Fundamentals of Data Science

Author
Matteo

November 2, 2026

Contents

1 Introduction **1**

1.1 What is Data Science? 1

1.2 The Data Science lifecycle 2

1.3 Artificial Intelligence and Machine Learning 3

1.4 Supervised models and prediction tasks 4

2 Data Representation **5**

2.1 The Iris dataset 5

2.2 Features and targets 6

2.3 Data types 6

2.4 Encoding categorical features 7

2.5 Labels 8

2.6 Dataset notation 9

2.7 Data generation and the i.i.d. assumption 10

2.8 Data points as vectors 11

2.9 Spanning sets 12

2.10 Basis 13

2.11 Standard basis 15

2.12 Coordinates in an orthonormal basis 18

2.13 Nonstandard bases 20

2.14 Finding the weights by least squares 23

2.15 Using fewer directions: subspaces 32

2.16 Projection onto a subspace 35

2.17 Orthonormal directions in a subspace 39

3 Principal Component Analysis **43**

3.1 Why learn directions from the data? 43

3.2 Choosing directions by reconstruction error 45

3.3 Measuring variation along a direction 48

3.4 Why maximum variance gives minimum error 49

4 Optimization **50**

4.1 What are we trying to optimize? 50

4.2 Global minima and maxima 51

4.3 Local minima and maxima 53

4.4 Why these are called zero-order conditions 54

4.5 Using the derivative to find candidate optima 55

4.6 Why a zero derivative is not enough 56

4.7 Several weights: a system of first-order equations 57

4.8 Approximating a function near one point 60

4.9 Choosing the direction of fastest change 63

4.10 Level curves and the gradient 65

5 Shallow Neural Networks **66**

5.1 Choosing features by hand 66

5.2 Learning the representation 67

5.3 Learning through several layers 68

5.4 A unifying view of models we already know 70

5.5 Why we need non-linear activations 74

5.6 An example shallow network 77

5.7 More hidden units 81

5.8 More outputs 82

5.9 How much can a network represent? 84

1 Introduction

Start of lecture 00 (Spinelli)

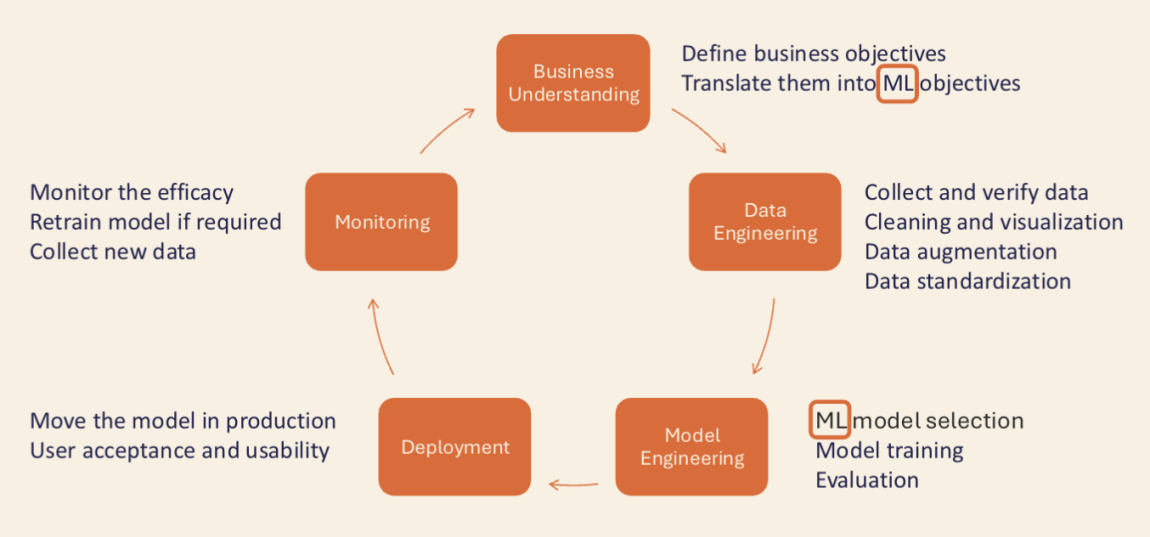
1.1 What is Data Science?

Definition 1.1: Data Science

Data Science is an interdisciplinary field that extracts knowledge from data to support decisions. It combines statistics, mathematics, computer science, and domain expertise to collect, clean, manage, and analyse data.

1.2 The Data Science lifecycle

A project moves through business understanding, data engineering, model engineering, deployment, and monitoring. This cycle can repeat when new data or retraining is needed.



The Data Science lifecycle. Source: Spinelli, Lecture 00.

1.3 Artificial Intelligence and Machine Learning

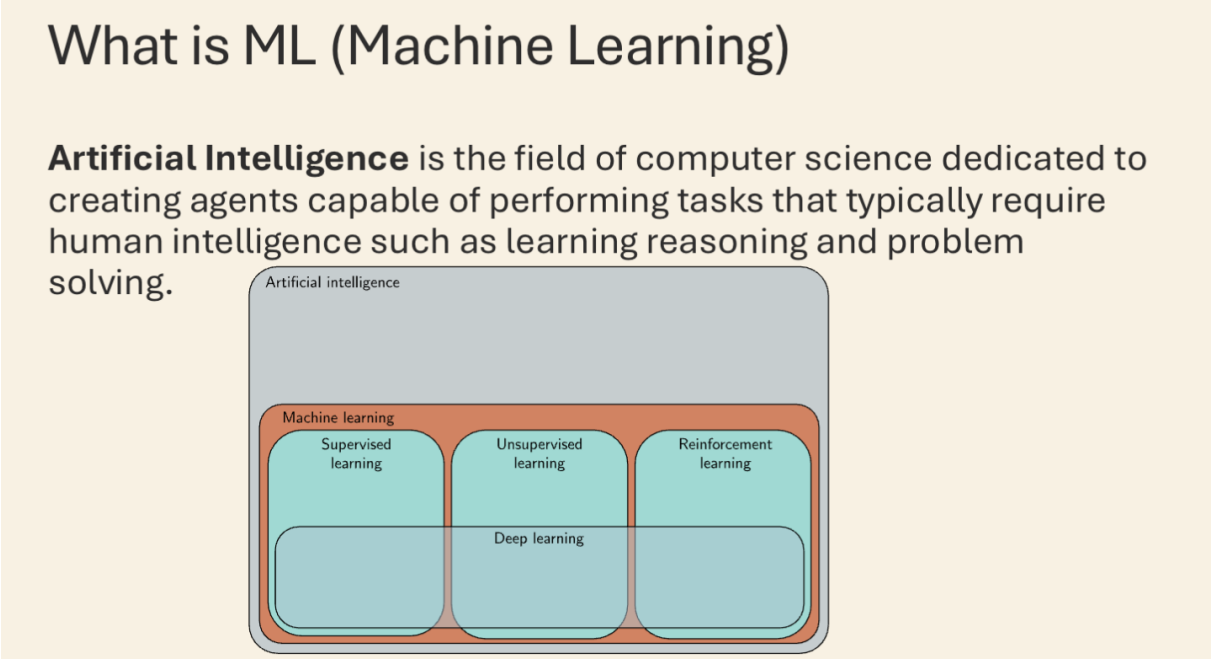
Artificial Intelligence (AI) studies how to build systems that perform tasks requiring abilities such as reasoning, learning, and problem solving.

Definition 1.2: Machine Learning

Machine Learning (ML) studies algorithms whose performance P on a task T improves through experience E. A learning problem therefore specifies the task, the performance measure, and the available experience.

The main learning settings are:

- **Supervised learning:** learn from inputs paired with known desired outputs to predict the output for new inputs.
- **Unsupervised learning:** discover structure in data without given target outputs.
- **Reinforcement learning:** learn how to act through interaction with an environment and feedback in the form of rewards.



AI, ML, and learning settings. Source: Spinelli, Lecture 00.

Deep learning uses neural networks with multiple layers and can be applied in each of these settings.

1.4 Supervised models and prediction tasks

Definition 1.3: Supervised learning model

A supervised learning model is a function $f : \mathcal{X} \to \mathcal{Y}$ that maps an input x to a predicted output $\hat{y} = f(x)$. Here, $\mathcal{X}$ is the input space and $\mathcal{Y}$ is the output space. Training means choosing or adjusting f using examples of inputs paired with their known outputs.

Definition 1.4: Regression

Regression is a supervised learning task whose target is a real-valued number or vector. For example, predicting a person's height from their age is a regression task.

Definition 1.5: Classification

Classification is a supervised learning task whose target is a category from a finite set of classes. For example, predicting whether a flower is setosa, versicolor, or virginica is a classification task.

2

Data Representation

Start of lecture 01 (Spinelli) — 2 October 2026

2.1 The Iris dataset

The Iris dataset contains 150 flowers, divided into three species: 50 setosa, 50 versicolor, and 50 virginica. The goal is to predict a flower's species from its measurements.

Example 1: Iris as a supervised classification task

Each **observation** is one flower, described by four **features**, all measured in centimetres:

- sepal length and sepal width;
- petal length and petal width.

The **target** is the flower's species. This is supervised learning because the species is known for the training examples, and classification because the target is a category.

2.2 Features and targets

In a tabular dataset, each row describes one observation. The feature columns contain its input measurements, while the target column contains the output we want to predict.

For the first flower shown in the lecture's table,

$$x = (4.3,\ 3.0,\ 1.1,\ 0.1), \qquad y = \text{setosa}.$$

The entries of x are, in order, sepal length, sepal width, petal length, and petal width, all in centimetres.

During training, both x and y are known. For a new input x, the model produces a prediction $\hat{y} = f(x)$. The observed target y and the predicted target $\hat{y}$ may differ.

2.3 Data types

Features and targets can have different types:

Type	**Values**	**Example**
Real-valued	$\mathbb{R}$	Petal length
Integer-valued	$\mathbb{Z}$	Number of purchases
Categorical	$\{C_1, \ldots, C_g\}$	Flower species
Binary	$\{0, 1\}$	Spam or non-spam

Predicting a numerical quantity is a regression task; predicting a category is a classification task. In Iris, the four features are numerical measurements and the target has three categories.

Assigning codes $0, 1, 2$ to the species does not turn them into numerical quantities: the codes identify categories, so the task remains classification.

2.4 Encoding categorical features

How can we give a colour to a model that works with numbers? Many learning algorithms require numerical features, so we need an *encoding*: a way to represent a category using numbers.

We could assign the codes red = 0, green = 1, and blue = 2. But a model that treats these codes as quantities could interpret blue as greater than green, or as twice as far from red as green is. Colours have no such order, and these numerical distances have no meaning here.

One-hot encoding replaces the colour column with three yes/no questions: "Is it red?", "Is it green?", and "Is it blue?". We use 1 for yes and 0 for no.

Example 2: Encoding a colour feature

Colour	Is it red?	Is it green?	Is it blue?
Red	1	0	0
Green	0	1	0
Blue	0	0	1

For example, a green product is represented by $(0, 1, 0)^\top$. Exactly one entry is 1 and all the others are 0: this is why the encoding is called *one-hot*.

A model predicting a product's price can then learn a separate effect for each colour, without assuming that blue is "twice" green. The cost is more columns: one feature with k categories becomes k numerical columns. We can now write this idea formally.

Definition 2.1: One-hot encoding

For a feature with k mutually exclusive categories, one-hot encoding represents category C_i by a vector $e_i \in \{0, 1\}^k$ whose i-th entry is 1 and whose other entries are 0:

$$(e_i)_j = \begin{cases} 1, & j = i, \\ 0, & j \neq i. \end{cases}$$

Each observation belongs to exactly one category, so its encoded vector has exactly one nonzero entry.

When categories have a natural order, an *ordinal encoding* can preserve it. For example, low, medium, and high can be coded as $0, 1, 2$. This expresses their order; it does not establish that the gaps between successive categories are equal.

2.5 Labels

Definition 2.2: Labels and labeled data

A *label* is an observed target value. In *labeled data*, the input and its target are both available.
In *unlabeled data*, the input is available but its target is unknown or unrecorded.

In Iris, a flower with measurements and a known species is labeled; the same measurements without the species are unlabeled. A label can also be numerical, as in a regression task.

2.6 Dataset notation

Let n be the number of observations and p the number of numerical features after any encoding. Write $\mathcal{X} \subseteq \mathbb{R}^p$ for the input space and $\mathcal{Y}$ for the target space.

Definition 2.3: Supervised dataset

A labeled dataset is an indexed collection of input–target pairs,

$$D = \big((x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\big) \in (\mathcal{X} \times \mathcal{Y})^n.$$

The i-th observation has an input column vector

$$x^{(i)} = (x_1^{(i)}, \ldots, x_p^{(i)})^\top \in \mathbb{R}^p$$

and a target $y^{(i)} \in \mathcal{Y}$.

The superscript (i) indexes an observation; the subscript j indexes a feature. Across all observations, the j-th feature column is

$$\mathbf{x}_j = (x_j^{(1)}, \ldots, x_j^{(n)})^\top \in \mathbb{R}^n.$$

Thus an input vector contains p features of one observation, while a feature column contains n values of one feature.

Example 3: A small supervised dataset

Suppose we predict house prices from area and room count. Here is a toy dataset:

House i	Area (m^2)	Rooms	Price (EUR)
1	50	2	150 000
2	80	3	240 000
3	100	4	300 000

Here $n = 3$ houses and $p = 2$ input features. The price is the target, so it is not counted in p. For the first house,

$$x^{(1)} = (50, 2)^\top, \qquad y^{(1)} = 150\,000.$$

D collects the three input–target pairs. Here $x_2^{(1)} = 2$ is the first house's room count. The rooms column is

$$\mathbf{x}_2 = (2, 3, 4)^\top.$$

The vector $x^{(1)}$ describes one house using both features; the vector $\mathbf{x}_2$ contains one feature for all three houses.

2.7 Data generation and the i.i.d. assumption

We model the observations as realizations of random pairs (X, Y) with an unknown joint probability distribution P_{XY} on $\mathcal{X} \times \mathcal{Y}$. Capital letters denote random variables; lowercase letters denote their observed values. Supervised learning uses these observations to learn a predictive relationship between input and target.

Definition 2.4: I.i.d. observations

The observations are *i.i.d.* when their random pairs $(X^{(1)}, Y^{(1)}), \ldots, (X^{(n)}, Y^{(n)})$ are mutually independent and all have the same distribution P_{XY}.

Independence is between observations; it does not require X and Y within one observation to be independent. The i.i.d. assumption can fail, for example when successive observations form a time series.

Note: Notation from the next section on

From the next section on we follow the notation of the slides, which comes from *Machine Learning Refined*. Each input has N coordinates and the dataset contains P points: compared with the notation above, $N = p$ and $P = n$.

- $x_p \in \mathbb{R}^N$ is the p-th point, for $p = 1, \ldots, P$.
- For a single vector without a point index, such as u or x, the subscripts $1, \ldots, N$ number its coordinates, as in the axis labels x_1, x_2 of the figures.
- When both indices are needed, $x_{n,p}$ is coordinate n of the point x_p.

2.8 Data points as vectors

We now focus on the inputs $x_p \in \mathbb{R}^N$. This space supports vector addition and multiplication by real numbers, called *scalars*. These operations act coordinate by coordinate:

$$u + v = (u_1 + v_1, \ldots, u_N + v_N)^\top, \qquad au = (au_1, \ldots, au_N)^\top.$$

An input can be drawn as a point with those coordinates or as a *position vector*: an arrow from the origin to that point.

2.8.1 Mean-centering

Definition 2.5: Mean-centered data

The sample mean input vector and the centered observations are

$$\bar{x} = \frac{1}{P}\sum_{p=1}^{P} x_p, \qquad \widetilde{x}_p = x_p - \bar{x}.$$

The mean of the centered inputs is zero: $\frac{1}{P}\sum_{p=1}^{P} \widetilde{x}_p = 0$.

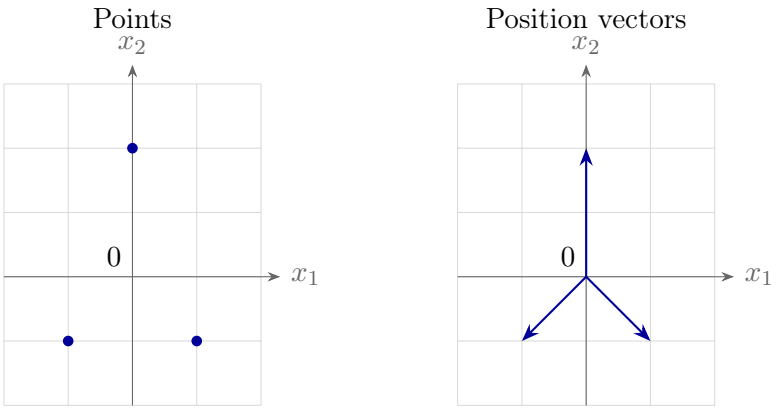
Centering subtracts each feature's sample mean from that feature. It is reversible if we retain $\bar{x}$, since $x_p = \widetilde{x}_p + \bar{x}$.

Example 4: Centering three points

For the inputs $(1,1)^\top$, $(3,1)^\top$, and $(2,4)^\top$,

$$\bar{x} = \left(\frac{1+3+2}{3}, \frac{1+1+4}{3}\right)^\top = (2,2)^\top.$$

The centered inputs are $(-1,-1)^\top$, $(1,-1)^\top$, and $(0,2)^\top$. Their sum is $(0,0)^\top$.



The same centered observations, drawn as points and as arrows.

2.9 Spanning sets

Definition 2.6: Span and spanning set

The *span* of vectors $c_1, \ldots, c_K$ is the set of all their linear combinations:

$$\operatorname{span}\{c_1, \ldots, c_K\} = \left\{ \sum_{n=1}^{K} a_n c_n : a_1, \ldots, a_K \in \mathbb{R} \right\}.$$

These vectors form a *spanning set* of a vector space V when their span equals V: every vector in V can be expressed using them.

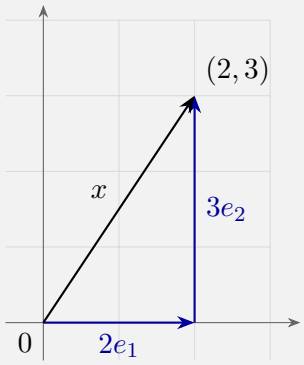
Example 5: Spanning the plane

The vectors

$$e_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad e_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

span $\mathbb{R}^2$, because every $(a, b)^\top$ can be written as $ae_1 + be_2$. For example,

$$\begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2e_1 + 3e_2.$$



Using only e_1 produces vectors $(a, 0)^\top$, so it spans the horizontal axis rather than the whole plane.

2.10 Basis

Definition 2.7: Linear independence

Vectors $c_1, \ldots, c_K \in \mathbb{R}^N$ are *linearly independent* if the only scalars $a_1, \ldots, a_K \in \mathbb{R}$ satisfying

$$a_1 c_1 + \cdots + a_K c_K = 0$$

are $a_1 = \cdots = a_K = 0$.

Definition 2.8: Basis

A *basis* of a vector space is a linearly independent spanning set. Equivalently, it is a minimal spanning set: removing any of its vectors destroys the spanning property.

Example 6: A spanning set with a redundant vector

The pair $\{e_1, e_2\}$ is a basis of $\mathbb{R}^2$. Adding $c_3 = e_1 + e_2$ still gives a spanning set, but it is not a basis, since

$$e_1 + e_2 - c_3 = 0$$

is a linear combination with nonzero coefficients. The third vector is redundant because the first two already generate it.

2.10.1 Representing observations in a basis

A basis of $\mathbb{R}^N$ contains exactly N vectors, each in $\mathbb{R}^N$. For a basis $c_1, \ldots, c_N$, every input observation has a unique representation

$$x_p = \sum_{n=1}^{N} w_{n,p} c_n, \qquad p = 1, \ldots, P.$$

The coefficients $w_{n,p}$ are the *weights*, or coordinates, of that observation in the chosen basis. The vector $w_p = (w_{1,p}, \ldots, w_{N,p})^\top$ records its representation; combining the basis vectors with these weights reconstructs x_p.

Example 7: Coordinates in a basis

Use the same basis for both observations:

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The two points have different weights:

$$x_1 = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2c_1 + 3c_2,$$

$$x_2 = \begin{pmatrix} 4 \\ 1 \end{pmatrix} = 4c_1 + c_2.$$

Observation	**Weight of c_1**	**Weight of c_2**
First ($p = 1$)	$w_{1,1} = 2$	$w_{2,1} = 3$
Second ($p = 2$)	$w_{1,2} = 4$	$w_{2,2} = 1$

For example, $w_{2,1} = 3$ means that the first observation uses three times the second basis vector. In $w_{n,p}$, the first index n identifies the basis vector; the second index p identifies the observation.

2.11 Standard basis

Definition 2.9: Standard basis

The *standard basis* of $\mathbb{R}^N$ consists of the vectors $c_1, \ldots, c_N$, where c_n has a 1 in position n and zeros everywhere else. In $\mathbb{R}^2$,

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Each vector points along one coordinate axis. Consequently, the weights in this basis are the entries of the observation itself:

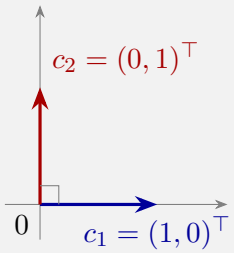
$$x_p = x_{1,p}c_1 + \cdots + x_{N,p}c_N \qquad \Longrightarrow \qquad w_p = x_p.$$

Example 8: Reading the weights from the coordinates

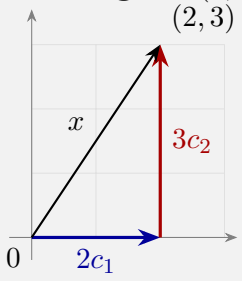
For $x = (2, 3)^\top$, move two units along the first axis and three along the second:

$$x = 2c_1 + 3c_2, \qquad w = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = x.$$

Unit basis vectors



Representing $x = (2, 3)^\top$



The blue arrow contributes the first coordinate; the red arrow contributes the second. Their sum is the black arrow.

2.11.1 Orthogonality, unit length, and the basis matrix

The standard basis has two properties:

- **Different vectors are perpendicular:** $c_n^\top c_m = 0$ for $n \neq m$.
- **Each vector has unit length:** $\|c_n\|_2^2 = c_n^\top c_n = 1$.

In $\mathbb{R}^2$, these follow directly from

$$c_1^\top c_2 = 1 \cdot 0 + 0 \cdot 1 = 0, \qquad \|c_1\|_2^2 = 1^2 + 0^2 = 1,$$

and similarly $\|c_2\|_2^2 = 1$.

Definition 2.10: Orthonormal basis

A basis is *orthonormal* if its vectors are mutually perpendicular and each has length 1.

How can we check whether a basis is orthonormal? We need to check the length of every vector and whether each pair of different vectors is perpendicular. With many vectors, drawing arrows is impractical. We can collect all these checks in one matrix.

First, place the basis vectors in the columns of C:

$$C = [\, c_1 \ \cdots \ c_N \,] \in \mathbb{R}^{N \times N}.$$

Now compute $C^\top C$. Each entry is a dot product between two columns of C: the entry in row n, column m is $c_n^\top c_m$. We read the result as a table of checks:

Position	What it tells us	Required value
On the diagonal	A vector's length squared	1
Off the diagonal	The dot product of two different vectors	0

If the diagonal contains only 1s and every other entry is 0, both checks pass: the basis is orthonormal. That result is the identity matrix I_N (written $I_{N\times N}$ in the slides), so the complete test is

$$\boxed{C^\top C = I_N.}$$

This formula is a compact way to check the lengths and perpendicularity of all the basis vectors at once. The standard basis passes the test.

Example 9: Using the orthonormality test

Consider the basis

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}, \qquad C = \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix}.$$

Both vectors have length 1, since $1^2 + 0^2 = 1$ and $0.6^2 + 0.8^2 = 1$. Do they also pass the perpendicularity check?

$$C^\top C = \begin{pmatrix} 1 & 0 \\ 0.6 & 0.8 \end{pmatrix} \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix} = \begin{pmatrix} 1 & 0.6 \\ 0.6 & 1 \end{pmatrix}.$$

The diagonal 1s confirm the unit lengths. The off-diagonal entries are 0.6, so the vectors are not perpendicular. Therefore $C^\top C \neq I_2$: this basis is not orthonormal, even though both vectors have length 1.

In the standard basis, the weights equal the entries of the observation: $w = x$. For a different orthonormal basis, the same observation can have different weights. The test $C^\top C = I_N$ checks the geometry of the basis; the chosen basis determines the weights.

2.12 Coordinates in an orthonormal basis

We now know how to check a basis. How do we find the weights needed to represent a point in that basis? For an orthonormal basis, a dot product with each basis vector gives the answer.

We know the observation x and choose the basis vectors $c_1, \ldots, c_N$. We want to find their weights $w_1, \ldots, w_N$ so that their weighted sum gives x. Put the basis vectors in the columns of C and the weights in w. Matrix multiplication performs that sum:

$$Cw = w_1 c_1 + \cdots + w_N c_N = x.$$

Thus $x = Cw$ is the compact form of a linear combination. Multiplying both sides by $C^\top$ gives

$$C^\top x = C^\top C w = I_N w = w.$$

Therefore the coordinates are

$$\boxed{w = C^\top x, \qquad w_n = c_n^\top x.}$$

The weight w_n tells us how far to move along the unit vector c_n. A positive weight follows its direction; a negative weight goes in the opposite direction. Perpendicular basis vectors do not contribute to this dot product, so it picks out just the weight we want.

For every observation x_p, the same calculation gives $w_p = C^\top x_p$. With a complete orthonormal basis, combining these weights with the basis vectors gives the original observation:

$$C w_p = C C^\top x_p = x_p.$$

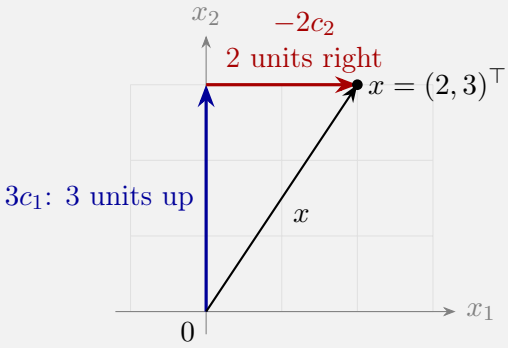
Here C is square and its columns form a basis of all of $\mathbb{R}^N$, so $CC^\top = I_N$ as well as $C^\top C = I_N$.

Example 10: Finding the weights of the point $(2, 3)$

Our observation is $x = (2, 3)^\top$: two units to the right and three units up. We want to describe this same point using two different unit directions:

$$c_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad \text{(up)}, \qquad c_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \quad \text{(left)}.$$

These vectors are perpendicular and have length 1, so they form an orthonormal basis. We seek weights w_1, w_2 for which $x = w_1 c_1 + w_2 c_2$.



Read the two moves in the drawing:

- Along c_1, we need to go up three units, so $w_1 = 3$.
- c_2 points left, but we need to go right two units. We move against its direction, so $w_2 = -2$.

The dot-product rule gives exactly these weights:

$$w_1 = c_1^\top x = 0 \cdot 2 + 1 \cdot 3 = 3,$$
$$w_2 = c_2^\top x = (-1) \cdot 2 + 0 \cdot 3 = -2.$$

Thus $w = (3, -2)^\top$. To check these weights, add their contributions:

$$Cw = 3c_1 - 2c_2 = \begin{pmatrix} 0 \\ 3 \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = x.$$

The sum gives our original observation. Its weights are $(2, 3)$ in the standard basis and $(3, -2)$ in the chosen basis. Both sets of weights describe the same point using their respective basis vectors.

2.13 Nonstandard bases

We can describe the same vector using a different basis. **The vector stays the same; its coordinates change.**

For a concrete example, let us choose the point $x = (5, 5)^\top$. Here x is simply the vector from the origin to that point. In the standard basis, its coordinates tell us to take 5 horizontal steps and 5 vertical steps:

$$x = 5\begin{pmatrix}1\\0\end{pmatrix} + 5\begin{pmatrix}0\\1\end{pmatrix}.$$

Now let us describe this same point using the two vectors from the slides:

$$c_1 = \begin{pmatrix}2\\1\end{pmatrix}, \qquad c_2 = \begin{pmatrix}1\\2\end{pmatrix}.$$

One step along c_1 moves us 2 units to the right and 1 up. One step along c_2 moves us 1 unit to the right and 2 up. The question is: **how many steps of each type do we need to reach the same point** $(5, 5)$**?** Fractional or negative steps are allowed.

These vectors form a basis because they are linearly independent: in the plane, they do not lie on the same line. We can therefore reach every point with exactly one pair of weights. A basis does not have to be orthonormal: these two vectors are neither perpendicular nor of unit length.

Example 11: Representing $(5, 5)$ in a different basis

Call the number of steps along c_1 and c_2 the weights w_1 and w_2. To reach our chosen point $x = (5, 5)^\top$, we need

$$w_1c_1 + w_2c_2 = x.$$

Matching the horizontal and vertical coordinates gives

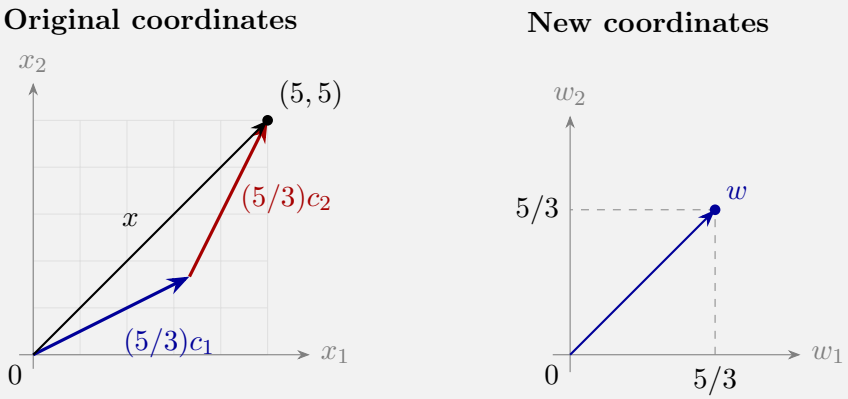
$$\begin{cases} 2w_1 + w_2 = 5, \\ w_1 + 2w_2 = 5. \end{cases}$$

Subtracting the second equation from the first gives $w_1 - w_2 = 0$, so the two weights are equal. Substituting into the first equation gives $3w_1 = 5$. Therefore,

$$w_1 = w_2 = \frac{5}{3}, \qquad x = \frac{5}{3}c_1 + \frac{5}{3}c_2.$$

We can check that these steps really reach our point:

$$\frac{5}{3}\begin{pmatrix}2\\1\end{pmatrix} + \frac{5}{3}\begin{pmatrix}1\\2\end{pmatrix} = \begin{pmatrix}10/3 + 5/3\\5/3 + 10/3\end{pmatrix} = \begin{pmatrix}5\\5\end{pmatrix}.$$



On the left, we follow the two contributions to reach the original point x. On the right, we record only how much of each basis vector we used: $w = (5/3, 5/3)^\top$. These are the new coordinates of the same vector; we have not moved the original point.

2.13.1 Encoding and decoding

We can go from a point to its weights, or from the weights back to the point. These two operations are called *encoding* and *decoding*.

Definition 2.11: Encoding and decoding

Encoding means starting from a point x and finding the weights w that describe it in the chosen basis.

Decoding means starting from those weights and adding the corresponding amounts of the basis vectors to recover x.

Let us follow both operations using the same example.

Example 12: From a point to its weights and back

Keep the two steps from the previous section:

$$c_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Encoding: from the point to the weights. Suppose we know the point we want to reach: $x = (5, 5)^\top$. How much of c_1 and how much of c_2 do we need? This is exactly the question we answered in the previous section:

$$x = \begin{pmatrix} 5 \\ 5 \end{pmatrix} \xrightarrow{\text{encoding}} w = \begin{pmatrix} 5/3 \\ 5/3 \end{pmatrix}.$$

The vector w simply collects our two answers: w_1 tells us how much of c_1 to use, and w_2 tells us how much of c_2 to use. Here we need $5/3$ of each. Finding these weights is *encoding*.

Decoding: from the weights back to the point. Now start from those same weights, $w = (5/3, 5/3)^\top$. They give us instructions: take $5/3$ of the first basis vector and add $5/3$ of the second. Following these instructions gives

$$x = \frac{5}{3}c_1 + \frac{5}{3}c_2 = \begin{pmatrix} 10/3 \\ 5/3 \end{pmatrix} + \begin{pmatrix} 5/3 \\ 10/3 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

This is *decoding*: we recover the point in its original coordinates. We have returned to exactly the point we started from:

$$\underbrace{(5,5)^\top}_{\text{original coordinates}} \xrightarrow{\text{encoding}} \underbrace{(5/3, 5/3)^\top}_{\text{weights in the new basis}} \xrightarrow{\text{decoding}} \underbrace{(5,5)^\top}_{\text{original coordinates}}.$$

Where does the matrix C come in? Put the two basis vectors in the columns of a matrix:

$$C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Multiplying C by the weights is a compact way to say "add the two steps":

$$Cw = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = w_1 c_1 + w_2 c_2.$$

So the same relation, $Cw = x$, describes both operations.

For **encoding**, we know x and find w by solving the system.

For **decoding**, we know w and calculate x by multiplying. The basis C tells us how to interpret the weights.

Example 13: Decoding a negative weight

Now start from a different pair of weights, $w = (-2, 1)^\top$. They give us instructions: take -2 times the first basis vector and add one copy of the second. Following these instructions gives

$$x = -2c_1 + c_2 = \begin{pmatrix} -4 \\ -2 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} -3 \\ 0 \end{pmatrix}.$$

This is decoding again: we recover the point in its original coordinates. These weights describe the point $x = (-3, 0)^\top$:

$$\underbrace{(-2,1)^\top}_{\text{weights in the new basis}} \xrightarrow{\text{decoding}} \underbrace{(-3,0)^\top}_{\text{original coordinates}}.$$

The negative weight means taking twice c_1 in the opposite direction: 4 units to the left and 2 down. Adding c_2 then moves us 1 unit to the right and 2 up, so we end at $(-3, 0)$.

Because the chosen vectors form a basis, every point has exactly one pair of weights, and encoding followed by decoding returns the same point.

2.14 Finding the weights by least squares

Which weights should we choose to reconstruct a point as well as possible? In the previous section, we could reach our point exactly. Now we introduce a way to measure how far off we are. This will let us choose the best weights even when reaching the point exactly is impossible.

Example 14: Trying weights and measuring the error

Let us keep the same point and basis as before:

$$c_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad x = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

Put c_1 and c_2 in the columns of C. Suppose we try the weights $w = (1, 1)^\top$: take one copy of each basis vector. The point we obtain is

$$Cw = c_1 + c_2 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}.$$

We wanted to reach $(5, 5)$, but reached $(3, 3)$. We are short by 2 units in each coordinate. Subtracting the desired point from the reconstructed point gives

$$Cw - x = \begin{pmatrix} 3 \\ 3 \end{pmatrix} - \begin{pmatrix} 5 \\ 5 \end{pmatrix} = \begin{pmatrix} -2 \\ -2 \end{pmatrix}.$$

This vector tells us how far off we are on each coordinate. To turn these errors into a single number, square them and add:

$$(-2)^2 + (-2)^2 = 8.$$

Call this number $h(w)$. It depends on the weights: changing w changes the point we reach, and therefore changes the error.

We already know the weights that reach our point exactly: $w = (5/3, 5/3)^\top$. With these weights, $Cw = x$, so

$$h(w) = 0^2 + 0^2 = 0.$$

We cannot do better than zero.

Why square the errors before adding them? Each squared error is nonnegative, so errors with opposite signs cannot cancel each other. We want a measure that is zero only when every coordinate is correct.

Definition 2.12: Reconstruction error and least squares

The **reconstruction error** is the vector $Cw - x$: the reconstructed point minus the original point.

The **squared reconstruction error** adds up the squares of its coordinates:

$$h(w) = \|Cw - x\|_2^2 = \sum_{n=1}^{N} \big((Cw)_n - x_n\big)^2.$$

Here $(Cw)_n$ is coordinate n of the reconstructed point and x_n is coordinate n of the original point.

Least squares means choosing the weights w that make this sum as small as possible.

The formula describes the same steps as the example: reconstruct the point, calculate the errors on its coordinates, square them, and add. With a complete basis, we can reach every point exactly and the minimum error is zero. But what if we can move in only one direction?

Example 15: Approximating a point with one direction

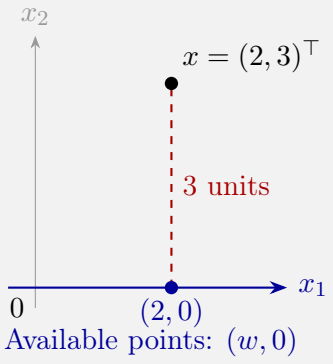
Let us choose a new point, $x = (2, 3)^\top$: two units to the right and three up. This time, suppose we can use only the horizontal vector

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

We have just one weight w, a single number. Whatever value we choose, we obtain

$$Cw = wc_1 = \begin{pmatrix} w \\ 0 \end{pmatrix}.$$

We can move right or left, but **the second coordinate always stays zero**. We therefore cannot reach $(2, 3)$ exactly.



We can still choose the closest point we can reach: $(2, 0)$, directly below the desired point. To get there, choose $w = 2$. Let us see why this also gives the smallest squared error:

$$h(w) = \underbrace{(w-2)^2}_{\text{squared horizontal error}} + \underbrace{(0-3)^2}_{\text{squared vertical error}} = (w-2)^2 + 9.$$

The 9 remains whatever weight we choose: we have no direction that lets us move upwards. We can, however, make the horizontal error zero by choosing $w = 2$:

$$h(2) = 0 + 9 = 9.$$

So **the best weight is 2, even though the minimum error is not zero**. Other weights add a horizontal error without reducing the vertical one.

We can also find this weight using the derivative. Since the constant 9 has derivative zero,

$$h'(w) = 2(w-2) = 0 \quad \Longrightarrow \quad w = 2.$$

This gives the same best weight we found by looking at the picture.

Least squares lets us find the best weights in both situations: when they give an exact reconstruction, and when they can only give an approximation.

2.14.1 The normal equations

How can we find the best weights without trying them one by one? In the previous example, we had just one weight. We found its best value by setting the derivative of the squared error to zero. With several weights, we need to check how the error changes with respect to each one.

Think of the weights as knobs: one controls how much of c_1 we use, another controls how much of c_2 we use, and so on. We study what happens when we turn one knob while keeping the others fixed.

Recall that a partial derivative changes one weight while holding the others fixed. The gradient collects these partial derivatives into one vector (see Definition 4.5). Setting it to zero means setting every one of those derivatives to zero.

For our squared error $h(w) = \|Cw - x\|_2^2$, when the columns of C are linearly independent, this procedure finds the unique minimum. Let us see what the calculations look like with two weights.

Example 16: Finding two weights using derivatives

Keep the same basis and point:

$$C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad x = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

For any choice of w_1 and w_2, the reconstructed point is

$$Cw = \begin{pmatrix} 2w_1 + w_2 \\ w_1 + 2w_2 \end{pmatrix}.$$

Subtract 5 from each coordinate, square the two errors, and add:

$$h(w_1, w_2) = (2w_1 + w_2 - 5)^2 + (w_1 + 2w_2 - 5)^2.$$

First, change w_1 while keeping w_2 fixed.

$$\begin{aligned} \frac{\partial h}{\partial w_1} &= 2(2w_1 + w_2 - 5) \cdot 2 + 2(w_1 + 2w_2 - 5) \cdot 1 \\ &= 2(5w_1 + 4w_2 - 15). \end{aligned}$$

Setting this to zero gives $5w_1 + 4w_2 = 15$.

Next, change w_2 while keeping w_1 fixed.

$$\begin{aligned} \frac{\partial h}{\partial w_2} &= 2(2w_1 + w_2 - 5) \cdot 1 + 2(w_1 + 2w_2 - 5) \cdot 2 \\ &= 2(4w_1 + 5w_2 - 15). \end{aligned}$$

Setting this to zero gives $4w_1 + 5w_2 = 15$.

We must satisfy both equations together:

$$\begin{cases} 5w_1 + 4w_2 = 15, \\ 4w_1 + 5w_2 = 15. \end{cases}$$

Subtracting the second equation from the first gives $w_1 - w_2 = 0$, so the weights are equal. The first equation becomes $9w_1 = 15$:

$$w_1 = w_2 = \frac{5}{3}.$$

These are the weights we already found before. They reconstruct $(5, 5)^\top$ exactly, so the minimum squared error is zero.

So far, our example had two coordinates and two weights. Before we generalize, let us see what changes when the number of chosen directions is smaller than the number of coordinates.

Example 17: Three coordinates, two chosen directions

Imagine a point in three-dimensional space:

$$x = \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}.$$

To reach it from the origin, move 2 units to the right, 3 forwards, and 4 up. The three available directions are

$$c_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \qquad c_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

Using all three, we can reach the point exactly:

$$x = 2c_1 + 3c_2 + 4c_3.$$

Now choose $K = 2$: keep only c_1 and c_2. The matrix of chosen directions is

$$C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

It has **3 rows**, because each point has three original coordinates, and **2 columns**, because we chose two directions. We started with three available directions; after choosing two, the matrix C we use has only two columns.

With two weights, we can reconstruct only points of the form

$$Cw = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} w_1 \\ w_2 \\ 0 \end{pmatrix}.$$

We can move right and forwards, but **we can no longer move up**: the third coordinate always stays zero.

To get as close as possible to $(2, 3, 4)^\top$, choose

$$w = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \qquad \hat{x} = Cw = \begin{pmatrix} 2 \\ 3 \\ 0 \end{pmatrix}.$$

The hat distinguishes the reconstructed point $\hat{x}$ from the original point x. Why are these weights the best? The squared error is

$$h(w_1, w_2) = (w_1 - 2)^2 + (w_2 - 3)^2 + (0 - 4)^2.$$

Choosing $w_1 = 2$ and $w_2 = 3$ makes the first two terms zero. The third term remains, whatever weights we choose:

$$h(2, 3) = 0 + 0 + 16 = 16.$$

We have described the point using **two weights instead of three coordinates**, but lost its height. Every point we can reconstruct lies in the horizontal plane where the third coordinate is zero.

So now we want to generalize. **If C has K columns, we need K weights**, one for each chosen direction. If the original points have N coordinates, C has N rows: multiplying an $N \times K$ matrix by a K-entry weight vector produces an N-entry reconstructed point. The three-coordinate example has $N = 3$ and $K = 2$.

With K weights, we set K partial derivatives to zero, obtaining one equation for each weight. We do not need to write out all the coordinate expressions every time: the derivative calculation has the compact matrix form

$$\nabla_w h(w) = 2C^\top (Cw - x).$$

Here $C^\top$ is the *transpose* of C: its rows are the columns of C. Setting the gradient to zero, dividing by 2, and expanding gives

$$\begin{aligned} C^\top (Cw - x) &= 0, \\ C^\top Cw - C^\top x &= 0, \\ C^\top Cw &= C^\top x. \end{aligned}$$

This last line is the system obtained from the derivatives, written in matrix form. It is called the *normal equations*.

Proposition 2.1: Normal equations

For a fixed matrix C with linearly independent columns, the unique weights that minimise $\|Cw - x\|_2^2$ satisfy

$$C^\top Cw = C^\top x.$$

We know C and x, so we can calculate $C^\top C$ and $C^\top x$. The only unknowns left are the weights in w.

Example 18: The same system in matrix form

Return to our two-dimensional basis $c_1 = (2, 1)^\top$, $c_2 = (1, 2)^\top$ and point $x = (5, 5)^\top$. Calculate the two known parts:

$$C^\top C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix},$$

$$C^\top x = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 5 \\ 5 \end{pmatrix} = \begin{pmatrix} 15 \\ 15 \end{pmatrix}.$$

Thus the normal equations say

$$\begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} 15 \\ 15 \end{pmatrix}.$$

Reading the first and second rows gives exactly the two equations we obtained by differentiating. Solving them gives $w = (5/3, 5/3)^\top$ again.

The entries of $C^\top C$ are the dot products between the chosen directions. If the columns of C are orthonormal, their dot products make $C^\top C = I$, and the equations simplify to $w = C^\top x$. This recovers the shortcut used earlier for orthonormal directions.

2.14.2 Encoding the whole dataset

So far, we have found the weights of one point. A dataset contains many points. **We now repeat the same operation for every point, using the same chosen directions.** The matrix C stays fixed; each point gets its own weights.

Definition 2.13: Encoding a dataset

To **encode a dataset** in fixed directions C, find a weight vector for each observation. Write x_p for observation p and w_p for its weights. Each reconstructed observation is Cw_p.

The subscript p identifies the observation. In particular, w_1 is now the *whole weight vector of the first observation*. Its individual entries are $w_{1,1}, \ldots, w_{K,1}$, one for each direction.

Example 19: Two points, the same basis

Keep our two basis vectors:

$$c_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Suppose our dataset contains just two observations:

$$x_1 = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \qquad x_2 = \begin{pmatrix} 6 \\ 6 \end{pmatrix}.$$

For the first point, one copy of each basis vector is enough:

$$c_1 + c_2 = \begin{pmatrix} 3 \\ 3 \end{pmatrix} \quad \Longrightarrow \quad w_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

For the second point, we need two copies of each:

$$2c_1 + 2c_2 = \begin{pmatrix} 6 \\ 6 \end{pmatrix} \quad \Longrightarrow \quad w_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

We have encoded both points. The directions did not change; the amount of each direction did.

We can also find these weights using the normal equations. For each observation p, solve

$$C^\top C w_p = C^\top x_p.$$

The left-hand matrix $C^\top C$ is the same for every point, so we calculate it once. The right-hand side changes because x_p changes.

Example 20: Finding each point's weights

Use the same two observations and basis as in the previous example. The shared matrix is

$$C^\top C = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}.$$

First observation: $x_1 = (3, 3)^\top$**.** Compute the right-hand side:

$$C^\top x_1 = \begin{pmatrix} 2 \cdot 3 + 1 \cdot 3 \\ 1 \cdot 3 + 2 \cdot 3 \end{pmatrix} = \begin{pmatrix} 9 \\ 9 \end{pmatrix}.$$

The system for this observation is

$$\begin{cases} 5w_{1,1} + 4w_{2,1} = 9, \\ 4w_{1,1} + 5w_{2,1} = 9. \end{cases}$$

Here the first subscript chooses the direction; the second chooses the observation. Subtracting the equations gives $w_{1,1} = w_{2,1}$. Substituting into the first gives $9w_{1,1} = 9$, hence

$$w_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad Cw_1 = \begin{pmatrix} 3 \\ 3 \end{pmatrix} = x_1.$$

Second observation: $x_2 = (6, 6)^\top$**.** Keep the same left-hand matrix, and calculate the new right-hand side:

$$C^\top x_2 = \begin{pmatrix} 2 \cdot 6 + 1 \cdot 6 \\ 1 \cdot 6 + 2 \cdot 6 \end{pmatrix} = \begin{pmatrix} 18 \\ 18 \end{pmatrix}.$$

The system is now

$$\begin{cases} 5w_{1,2} + 4w_{2,2} = 18, \\ 4w_{1,2} + 5w_{2,2} = 18. \end{cases}$$

Subtracting again shows that the weights are equal. The first equation gives $9w_{1,2} = 18$, hence

$$w_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \qquad Cw_2 = \begin{pmatrix} 6 \\ 6 \end{pmatrix} = x_2.$$

We obtain the same weights as by inspecting the two points.

How well do we reconstruct the whole dataset? Calculate the squared error for each observation, add these errors, and divide by the number of observations.

Definition 2.14: Average squared reconstruction error

For P observations, the average squared reconstruction error is

$$\frac{1}{P}\sum_{p=1}^{P}\|Cw_p - x_p\|_2^2.$$

Each term measures the squared error for one observation. The sum adds the errors of all P observations, and dividing by P gives their average.

Example 21: Average error for our two observations

Both reconstructions above are exact: $Cw_1 = x_1$ and $Cw_2 = x_2$. Their squared errors are therefore both zero, so the average is

$$\frac{0+0}{2} = 0.$$

For fixed C, changing one observation's weights affects only that observation's error. Finding the best weights for each point separately therefore also minimises the average error.

With a complete basis, each point still has as many weights as original coordinates. To describe it with fewer numbers, we must choose fewer directions. The next section explores this idea.

2.15 Using fewer directions: subspaces

We have learned how to encode every point using the same directions. But with a complete basis, a point with two original coordinates still needs two weights. We have changed its description without reducing the number of values we store for each point.

Could we describe the same observations using fewer numbers? To try this, we choose fewer directions. Let us return to the small dataset we have just encoded.

Example 22: Two coordinates, but only one weight

Our two observations are

$$x_1 = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \qquad x_2 = \begin{pmatrix} 6 \\ 6 \end{pmatrix}.$$

Notice what they have in common: in each point, the two coordinates are equal. Once we know one coordinate, we already know the other. Storing both repeats the same information.

Instead of the two directions used before, choose just the diagonal vector

$$c_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

One step along this vector moves us 1 unit to the right and 1 up. To reach the first point, take three steps. To reach the second, take six:

$$x_1 = 3c_1, \qquad x_2 = 6c_1.$$

We can therefore encode the first observation with the single weight $w_1 = 3$ and the second with the single weight $w_2 = 6$. Here the subscript identifies the observation; each weight is now just one number.

We keep the chosen direction once for the whole dataset. For each observation, we then need only one number instead of two. Knowing that direction, we can still reconstruct both coordinates exactly:

$$3\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \qquad 6\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \end{pmatrix}.$$

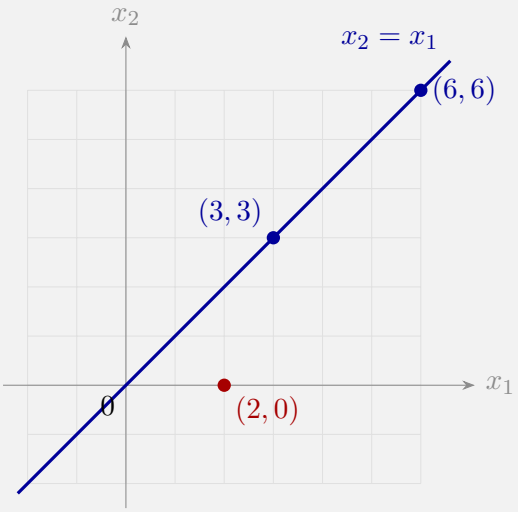
What have we given up by keeping only one direction? We can still reach both observations in this dataset, but we can no longer reach every point in the plane.

Example 23: Which points can our one direction reach?

Keep $c_1 = (1, 1)^\top$. The matrix C has just this one column. With any real weight w, we reconstruct

$$Cw = w \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} w \\ w \end{pmatrix}.$$

Every reachable point has equal coordinates. Together, these points form the diagonal line $x_2 = x_1$ through the origin. Positive weights take us to the upper right; negative weights take us to the lower left. Weight zero gives the origin. We can move along the whole line, in both directions.



Now try to reach a different point, $x = (2, 0)^\top$. Its first coordinate would require $w = 2$, but its second would require $w = 0$. We have only one weight, so we cannot satisfy both requirements. This point lies outside our line.

The line is our collection of reachable points. It has a name: a *subspace*. The original observations still have two coordinates, but moving within this line needs only one independent number. That is why the line has dimension one.

Definition 2.15: Subspace spanned by the columns

The **span** of the columns of C contains all their linear combinations:

$$S = \text{span}\{c_1, \ldots, c_K\} = \{Cw : w \in \mathbb{R}^K\}.$$

These points form a **linear subspace**: a set containing the origin and closed under vector addition and multiplication by real numbers.

If its K columns are independent, they form a basis of S, and S has **dimension** K.

From the picture to N coordinates and K directions. Each original observation has N coordinates. Choose $K < N$ linearly independent directions and put them in the

columns of C:

$$\underbrace{C}_{N\times K}\ \underbrace{w}_{K\text{ weights}} = \underbrace{Cw}_{N\text{ coordinates}}.$$

The directions still have N entries each. Choosing fewer of them reduces the number of weights, not the number of coordinates in a reconstructed point. All reconstructions lie in the same K-dimensional subspace S.

In the diagonal example, $N = 2$ and $K = 1$: our reachable set is a line. In the earlier three-coordinate example, $N = 3$ and $K = 2$: keeping the two horizontal directions gave a plane through the origin. In each case, K tells us how many independent weights we can adjust.

Definition 2.16: Dimensionality reduction

Dimensionality reduction represents observations with fewer numbers. Here, we replace each observation's N original coordinates with $K < N$ weights in shared, linearly independent directions.

A point inside their span can be reconstructed exactly. A point outside it cannot: we must settle for an approximation.

What happens to a point outside our subspace? For our diagonal line, $(3, 3)^\top$ and $(6, 6)^\top$ are recovered exactly. The point $(2, 0)^\top$ cannot be. We already know from least squares what to do: choose the reachable point with the smallest error. Next, we find that point on the line and give this operation its geometric name: *projection*.

2.16 Projection onto a subspace

At the end of the previous section, we had a problem: our chosen direction let us move along a diagonal line, but the point we wanted to describe was outside that line.

We cannot reach the original point exactly. We can still ask: **which point on our line gets us closest?** Let us find it, then look at what this choice means geometrically.

Example 24: Finding the closest point on the diagonal

Keep the direction $c_1 = (1,1)^\top$ and the point $x = (2,0)^\top$. Our matrix C has one column, so we have just one weight w:

$$C = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad Cw = \begin{pmatrix} w \\ w \end{pmatrix}.$$

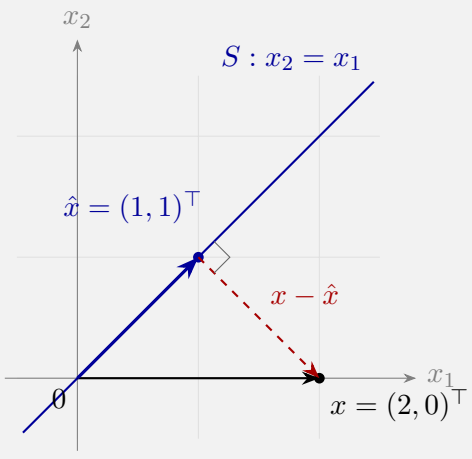
Every value of w gives a point on the line $x_2 = x_1$. No value reaches $(2,0)$, because its two coordinates are different. To choose the closest point, minimise the squared error:

$$h(w) = (w-2)^2 + w^2 = 2(w-1)^2 + 2.$$

The term $2(w-1)^2$ is nonnegative, and becomes zero at $w = 1$. So the best weight is 1, and the minimum squared error is 2.

Write the best weight as w_* and the reconstructed point as $\hat{x}$. The star marks our best choice; the hat distinguishes the reconstruction from the original point:

$$w_* = 1, \qquad \hat{x} = Cw_* = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$



The blue line contains every point we can reconstruct. The black point is the original observation. The blue point is our closest reconstruction. The dashed segment joins these last two points; notice that it meets the line at a right angle.

We have replaced $(2,0)^\top$ with $(1,1)^\top$, the best point we can reach using our chosen direction. This is an *orthogonal projection* onto the line.

The same idea works with a plane or a higher-dimensional subspace: among all the points our chosen directions can reach, keep the one closest to the original observation.

Definition 2.17: Orthogonal projection

The **orthogonal projection** of x onto a linear subspace S is the point $\hat{x}$ in S closest to x.

If the linearly independent columns of C span S, write

$$w_* = \arg\min_w \|Cw - x\|_2^2, \qquad \hat{x} = Cw_*.$$

The notation arg min returns the *weights* giving the smallest error. Multiplying by C then gives the *point* in the subspace.

How do we find the projection with several directions? Use the normal equations we already know:

$$C^\top C w_* = C^\top x.$$

First solve for the best weights w_*. Then decode them by computing $\hat{x} = Cw_*$. Nothing here requires the minimum error to be zero. The equations find the closest reconstruction even when the original point lies outside the subspace.

Example 25: Finding the point with normal equations

For $C = (1, 1)^\top$ and $x = (2, 0)^\top$, the two known quantities are

$$C^\top C = 1 \cdot 1 + 1 \cdot 1 = 2, \qquad C^\top x = 1 \cdot 2 + 1 \cdot 0 = 2.$$

So the normal equations reduce to $2w_* = 2$, giving $w_* = 1$. Decoding gives $\hat{x} = (1, 1)^\top$, the same closest point as before.

There is also a compact formula. Independent columns make $C^\top C$ invertible. Multiplying the normal equations on the left by $(C^\top C)^{-1}$ gives

$$\underbrace{(C^\top C)^{-1}(C^\top C)}_{I_K} w_* = (C^\top C)^{-1} C^\top x.$$

Thus the two steps, finding the weights and reconstructing the point, can be written as

$$w_* = (C^\top C)^{-1} C^\top x, \qquad \hat{x} = C(C^\top C)^{-1} C^\top x.$$

These are compact expressions for the same operations. We can find the weights by solving the normal equations directly.

2.16.1 Why the projection is perpendicular

Look again at the dashed segment from the reconstructed point to the original point. It represents what is left over after using our chosen directions.

Definition 2.18: Residual

The **residual** is the vector from the reconstruction to the original point:

$$r = x - \hat{x} = x - Cw_*.$$

It has the opposite sign to the earlier error vector $Cw_* - x$, but the same squared length. The squared reconstruction error is therefore $\|r\|_2^2$.

Why does this vector meet the subspace at a right angle? The normal equations give us the answer:

$$C^\top r = C^\top x - C^\top C w_* = 0.$$

Each row of $C^\top$ is one chosen direction transposed. This equation therefore says that $c_k^\top r = 0$ for every chosen direction c_k. A zero dot product means perpendicularity: r is perpendicular to each direction, and hence to every vector in their span. If $r = 0$, the reconstruction is already exact.

Example 26: Checking the right angle

For our diagonal line, $\hat{x} = (1, 1)^\top$ and $x = (2, 0)^\top$. The part left over is

$$r = x - \hat{x} = \begin{pmatrix} 2 \\ 0 \end{pmatrix} - \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Check its dot product with the direction of the line:

$$c_1^\top r = (1, 1) \begin{pmatrix} 1 \\ -1 \end{pmatrix} = 1 - 1 = 0.$$

The residual is perpendicular to the line. Its squared length is $1^2 + (-1)^2 = 2$, matching the minimum squared error we found earlier.

Imagine choosing another point y in the subspace. Starting from $\hat{x}$, reaching y adds a sideways displacement within S. That displacement is perpendicular to r. By Pythagoras,

$$\|x - y\|_2^2 = \|x - \hat{x}\|_2^2 + \|\hat{x} - y\|_2^2.$$

Any different point adds a positive term to the distance squared. This is why the perpendicular projection is the closest point.

2.16.2 Encoding and decoding the projection

The two operations still follow the same order: find the weights, then use them to reconstruct a point. The change is that decoding now returns the closest point in S, which may differ from x.

Example 27: Encoding and decoding one point

Start with our original point $x = (2, 0)^\top$ and the fixed direction $C = (1, 1)^\top$.

Encoding. Solve the normal equations $2w_* = 2$. We obtain and store just one number: $w_* = 1$.

Decoding. Follow that weight's instruction: take one copy of the chosen direction. This gives

$$\hat{x} = C \cdot 1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

The full journey is

$$\underbrace{\begin{pmatrix} 2 \\ 0 \end{pmatrix}}_{\text{original: 2 coordinates}} \xrightarrow{\text{encode}} \underbrace{1}_{\text{1 weight}} \xrightarrow{\text{decode}} \underbrace{\begin{pmatrix} 1 \\ 1 \end{pmatrix}}_{\text{reconstructed: 2 coordinates}} .$$

We return to two coordinates, but we do not recover the original point: the residual $r = (1, -1)^\top$ was left out.

Suppose instead we started from $(1, 1)^\top$, already on the line. It would also be encoded with weight 1, and decoding would return it exactly. The same stored weight can therefore come from different original points. It records where we end up on the line, without recording the part perpendicular to it.

With K chosen independent directions in N dimensions, encoding stores K weights and decoding produces N coordinates in their span. Points already in that span are recovered exactly; other points are replaced by their projections.

2.17 Orthonormal directions in a subspace

We now know how to find the closest point in a chosen subspace: solve the normal equations, then reconstruct the point using the weights. Can we make the calculation simpler?

Yes: choose directions that are **mutually perpendicular and have length one**. These are the orthonormal directions introduced in Definition 2.10. We can use them as a basis of a smaller subspace, without needing a basis of the whole original space.

Let us first return to our diagonal line. We can change the length of the vector we use to describe it without changing the line itself.

Example 28: The same line, a unit direction

We still want to approximate $x = (2, 0)^\top$ on the line $x_2 = x_1$. Previously we used the direction $(1, 1)^\top$. Its length is $\sqrt{1^2 + 1^2} = \sqrt{2}$. Divide by that length to get a unit vector:

$$c_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad C = c_1.$$

This vector points along the same diagonal, but one copy of it is a shorter step than one copy of $(1, 1)^\top$.

With a unit direction, its dot product with x gives the best weight:

$$w_* = c_1^\top x = \frac{1}{\sqrt{2}} \cdot 2 + \frac{1}{\sqrt{2}} \cdot 0 = \sqrt{2}.$$

To decode, take $\sqrt{2}$ copies of the new direction:

$$\hat{x} = Cw_* = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \sqrt{2} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

We reach the same projected point as before. The old direction needed weight 1; the shorter unit direction needs weight $\sqrt{2}$. **The weight changes because the size of one step changes.** The line and the closest point stay the same.

Why does the dot product give the weight directly? Recall that $C^\top C$ collects the dot products between the columns of C. With orthonormal columns, each vector's dot product with itself is 1, and the dot product of two different columns is 0. For K columns, this gives $C^\top C = I_K$.

Substitute this into the normal equations:

$$C^\top C w_* = C^\top x \quad \Longrightarrow \quad I_K w_* = C^\top x \quad \Longrightarrow \quad w_* = C^\top x.$$

There is no system left to solve: each weight can be calculated separately as $w_{*,k} = c_k^\top x$. It measures the signed amount of x along unit direction c_k.

Proposition 2.2: Orthonormal projection formulas

If the columns of $C \in \mathbb{R}^{N \times K}$ are orthonormal, the best weights and closest reconstruction are

$$\underbrace{w_* = C^\top x}_{\text{encoding}}, \qquad \underbrace{\hat{x} = C w_* = C C^\top x}_{\text{decoding}}.$$

Encoding takes a dot product with each chosen direction. Decoding multiplies each direction by its weight and adds the contributions.

The two matrix products have different jobs. The order matters. $C^\top C$ acts on the K weights and equals I_K. The product $CC^\top$ acts on the original N coordinates: it first encodes the point and then decodes its weights.

Definition 2.19: Orthogonal projection matrix

For orthonormal columns of C, the matrix $CC^\top$ is the **orthogonal projection matrix** onto their span S. Multiplying an original point by it returns the closest point in S:

$$\hat{x} = CC^\top x.$$

Example 29: Two products, two different results

For our unit diagonal direction,

$$C = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

has two rows and one column. The product $C^\top C$ is a single entry:

$$C^\top C = \frac{1}{2} + \frac{1}{2} = 1.$$

This is I_1, confirming that the direction has unit length. But reversing the order produces a 2×2 matrix:

$$CC^\top = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

It is not I_2. It projects points onto our line:

$$CC^\top \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

This multiplication combines the two operations we just performed: find the weight $\sqrt{2}$, then use it to reconstruct $(1, 1)^\top$.

Orthonormality simplifies the calculation; it does not add any missing directions. If $K < N$, the projection can still leave out part of the original point. With a complete orthonormal basis ($K = N$), every point can be recovered, and $CC^\top = I_N$ as well.

2.17.1 From three coordinates to two

Let us apply the shortcut to our earlier three-coordinate example. We kept two directions and lost the height of the point. Those directions were already perpendicular unit vectors, so we can now find their weights directly by dot products.

Example 30: Two weights for a point in space

Our point is $x = (2, 3, 4)^\top$: 2 units to the right, 3 forwards, and 4 up. Keep only the first two directions:

$$c_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \qquad C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

They are perpendicular and each has length 1. Their combinations form the horizontal plane $x_3 = 0$.

Encoding: find the two weights. Take the dot product with each direction:

$$w_{*,1} = c_1^\top x = 1 \cdot 2 + 0 \cdot 3 + 0 \cdot 4 = 2,$$
$$w_{*,2} = c_2^\top x = 0 \cdot 2 + 1 \cdot 3 + 0 \cdot 4 = 3.$$

Together, these give

$$w_* = C^\top x = \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

We store two numbers, telling us to use c_1 twice and c_2 three times.

Decoding: follow those instructions.

$$\hat{x} = Cw_* = 2c_1 + 3c_2 = \begin{pmatrix} 2 \\ 3 \\ 0 \end{pmatrix}.$$

The reconstructed point has three coordinates again, but its height is zero. The directions we kept cannot move us upwards. The part left out is

$$r = x - \hat{x} = \begin{pmatrix} 0 \\ 0 \\ 4 \end{pmatrix}, \qquad \|r\|_2^2 = 16.$$

This is the same projection and error we found before. Orthonormality has made finding the weights easier.

Here the two matrix products are especially easy to read:

$$C^\top C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I_2, \qquad CC^\top = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

The projection matrix keeps the first two coordinates and replaces the third with zero.

2.17.2 What changes when we reduce the dimension?

There are two separate choices. **How many directions we keep** determines which points we can reconstruct exactly. **Whether those directions are orthonormal** determines whether we can use the dot-product shortcut.

For linearly independent columns, the cases are

Directions	**Finding the weights**	**Reconstruction**
Complete basis	Solve the normal equations.	Exact for every point.
Complete, orthonormal	$w_* = C^\top x$.	Exact for every point.
Basis of a subspace	Solve the normal equations.	Closest point in S.
Orthonormal in a subspace	$w_* = C^\top x$.	Closest point in S.

With fewer directions, points inside S are still recovered exactly. Points outside it are replaced by their projections; the perpendicular residual is lost. Averaging the squared lengths of these residuals gives the dataset's average squared reconstruction error.

So far, we have chosen the directions first and then found the best weights. Next, we ask how to choose the directions themselves from the data.

Source: Spinelli, Lecture 01 (data_vectors), slides 18–35. The slide deck credits Machine Learning Refined, Chapter 8.

3

Principal Component Analysis

Start of lecture 02 (Spinelli)

3.1 Why learn directions from the data?

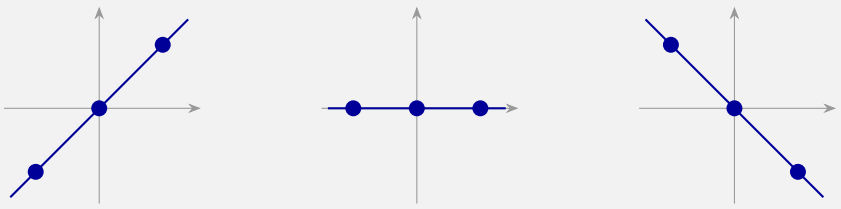
We have learned how to describe points using fewer directions. Until now, we chose those directions first, then found the best weights for each point. But one question remains: **which directions should we choose?**

The observations themselves can give us a clue. If they follow a particular pattern, a direction that follows that pattern may let us describe them with fewer numbers while losing little information. Let us look at small datasets where we can see this immediately.

Example 31: Three points, one line

In the previous chapter the direction was given, and we only had to find the weights. Now we are given only the observations. Each has two features, and we may keep just **one** direction. Which one should it be?

Look at three small datasets, all already centred (their mean is $(0, 0)^\top$):



(a) (b) (c)

	Observations	Direction to keep
(a)	$(-2,-2)^\top,\ (0,0)^\top,\ (2,2)^\top$	$(1,1)^\top$, the diagonal
(b)	$(-2,0)^\top,\ (0,0)^\top,\ (2,0)^\top$	$(1,0)^\top$, the horizontal axis
(c)	$(-2,2)^\top,\ (0,0)^\top,\ (2,-2)^\top$	$(1,-1)^\top$, the anti-diagonal

In each dataset the three points lie on one line through the origin. If we keep the direction of that line, every point is reached exactly with one weight, as in Example 22. Any other direction would leave a residual for the nonzero points.

The question is the same each time, but the answer changes with the data. No direction is good in general: the good one is the direction along which *these* observations move.

From now on we keep dataset (a), so $x_1 = (-2,-2)^\top$, $x_2 = (0,0)^\top$, $x_3 = (2,2)^\top$. With the unit diagonal $c_1 = \frac{1}{\sqrt{2}}(1,1)^\top$ of Example 28, their weights are $w_p = c_1^\top x_p$, that is $-2\sqrt{2}$, 0, and $2\sqrt{2}$.

Here we could find the direction by eye, because the points lie *exactly* on a line. Real observations usually lie only *close* to a line or plane. Then every direction loses some information, and looking at the picture is no longer enough: we need a number to compare the candidates.

Principal Component Analysis (PCA) makes this choice from the data. It looks for the directions that make the average squared reconstruction error as small as possible. We will write this choice precisely in the next section.

What if the cloud is away from the origin? The lines and planes we have used so far pass through the origin. A real cloud of observations can sit elsewhere. Before choosing PCA directions, we move the cloud so that its mean is at the origin. This is the mean-centring introduced in Definition 2.5. It preserves the differences between observations while removing their common offset.

Example 32: The same pattern away from the origin

Suppose the original observations are now

$$\begin{pmatrix} 8 \\ 18 \end{pmatrix}, \qquad \begin{pmatrix} 10 \\ 20 \end{pmatrix}, \qquad \begin{pmatrix} 12 \\ 22 \end{pmatrix}.$$

Their mean is $\mu = (10, 20)^\top$. Subtracting μ from each point gives exactly the centred observations of dataset (a) in Example 31. For the last point,

$$x_3 = \begin{pmatrix} 12 \\ 22 \end{pmatrix} - \begin{pmatrix} 10 \\ 20 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

We can encode this centred point with the same diagonal direction and weight $w_3 = 2\sqrt{2}$. Decoding gives $(2, 2)^\top$. To return to the original location, add the mean back:

$$\mu + c_1 w_3 = \begin{pmatrix} 10 \\ 20 \end{pmatrix} + \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 22 \end{pmatrix}.$$

The mean and direction are kept once for the dataset; each observation still needs only its own weight. All the x_p used in the following PCA calculations refer to the centred observations.

We can now separate the two choices clearly: for any fixed directions, we know how to find the best weights. PCA also chooses the directions. Next, we compare those choices through their reconstruction errors.

3.2 Choosing directions by reconstruction error

In the previous section, the diagonal looked like a good direction because it followed the pattern in our data. Now we want a numerical way to compare it with other choices.

The idea is to **reconstruct the same observations using each candidate set of directions, then compare the errors**. For each choice, we use the best possible weights. This way, we judge the directions themselves, rather than a poor choice of weights.

Example 33: The same data, two choices of direction

Keep the three centred observations from Example 31:

$$x_1 = \begin{pmatrix} -2 \\ -2 \end{pmatrix}, \qquad x_2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \qquad x_3 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

We want just one weight per observation, so we keep one direction. Compare these two unit vectors:

$$c_h = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{(horizontal)}, \qquad c_d = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{(diagonal)}.$$

For each choice, C contains just that one column. Both choices use the same number of weights; the direction is what changes.

First, try the horizontal direction. For the point $x_3 = (2, 2)^\top$, the best weight is

$$w_3 = c_h^\top x_3 = 2.$$

Decoding takes two horizontal steps:

$$\hat{x}_3 = c_h w_3 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}.$$

We kept the first coordinate but lost the second. The squared error is

$$\|\hat{x}_3 - x_3\|_2^2 = (2-2)^2 + (0-2)^2 = 4.$$

The negative point also loses its second coordinate and has squared error 4. The origin is reconstructed exactly, so its error is 0.

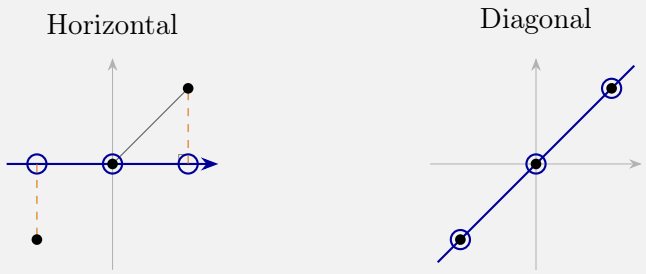
Now try the diagonal direction. For the same point, the best weight is

$$w_3 = c_d^\top x_3 = 2\sqrt{2}.$$

Decoding gives

$$\hat{x}_3 = c_d w_3 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} 2\sqrt{2} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

This reconstruction is exact. The other two observations also lie on the diagonal, so their squared errors are zero too.



Black dots are the original observations; blue circles are their reconstructions. Dashed segments show the parts we lose. With the diagonal choice, every blue circle sits on its original black dot.

To compare the choices using the whole dataset, average the squared errors of all three observations:

Observation	Horizontal error	Diagonal error
$(-2,-2)^\top$	4	0
$(0,0)^\top$	0	0
$(2,2)^\top$	4	0

The entries are squared errors. Their averages are

$$\text{horizontal: } \frac{4+0+4}{3} = \frac{8}{3}, \qquad \text{diagonal: } \frac{0+0+0}{3} = 0.$$

The diagonal wins: it uses one weight per observation, just like the horizontal choice, but reconstructs every observation exactly.

What does PCA choose for a larger dataset? The same comparison applies when the points have many features and we keep more than one direction. Write P for the number of observations, N for their original number of coordinates, and $K < N$ for the number of directions we keep.

Put the shared directions in the columns of $C \in \mathbb{R}^{N \times K}$. Each centred observation x_p gets its own K-entry weight vector w_p. Decoding gives Cw_p, which we compare with x_p.

Definition 3.1: Principal Component Analysis

For mean-centred observations, **Principal Component Analysis** chooses K perpendicular unit directions and each observation's weights to minimise the average squared reconstruction error:

$$\min_{C, w_1, \ldots, w_P} \frac{1}{P} \sum_{p=1}^{P} \|Cw_p - x_p\|_2^2, \qquad C^\top C = I_K.$$

The condition $C^\top C = I_K$ requires the chosen columns to be orthonormal. PCA is **unsupervised**: it uses the input observations without needing target labels.

Read the objective as the procedure we just followed: reconstruct each point, square its errors on the original coordinates, add those errors across the dataset, and divide by P to get the average. The minimisation asks which directions and weights give the smallest result. A point cloud close to a line or plane may have a positive minimum; the exact reconstruction in our example is a special case.

For fixed directions, we already know the best weights. Once we choose an orthonormal C, the previous chapter gives

$$w_p = C^\top x_p, \qquad \hat{x}_p = Cw_p = CC^\top x_p.$$

So we do not need to search for the weights independently. For each candidate C, calculate them with dot products, reconstruct the observations, and measure the average error.

Substituting these best weights into the objective gives a score that depends only on the chosen directions:

$$g(C) = \frac{1}{P} \sum_{p=1}^{P} \|CC^\top x_p - x_p\|_2^2, \qquad C^\top C = I_K.$$

Here $g(C)$ is the average squared reconstruction error after using that choice of directions as well as possible. Change C, and both the best weights and the reconstructions change with it. PCA seeks the smallest score among all valid choices of C.

We now have a way to judge directions by what they lose. Next, we look at how much the observations spread along a direction, and connect that spread to the reconstruction error.

3.3 Measuring variation along a direction

We now connect reconstruction error to how much the observations spread along a chosen direction. First consider keeping just one direction c, so $C = c$ and each encoding is a single number $w_p = c^\top x_p$. We compare unit vectors: otherwise simply doubling c would double the coordinates and make their spread look larger without changing the direction.

Definition 3.2: Variance along a unit direction

Let c have length 1. Because the observations are centred, their encoded coordinates $w_p = c^\top x_p$ also have mean 0. Their empirical variance, with the $1/P$ convention used here, is

$$V(c) = \frac{1}{P}\sum_{p=1}^{P} w_p^2 = \frac{1}{P}\sum_{p=1}^{P}\left(c^\top x_p\right)^2.$$

Variance is the average squared deviation from the mean. Here that mean is zero, so each deviation is the encoded coordinate itself. Larger $V(c)$ means greater spread.

Example 34: Computing the spread in the two directions

Use the same three observations and compare the unit directions

$$c_h = \begin{pmatrix}1\\0\end{pmatrix}, \qquad c_d = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix}.$$

The diagonal vector $(1,1)^\top$ has length $\sqrt{2}$. Dividing by $\sqrt{2}$ makes its length 1, so both choices use the same unit of distance. For $x_3 = (2,2)^\top$, its diagonal coordinate is

$$w_3 = c_d^\top x_3 = \frac{2}{\sqrt{2}} + \frac{2}{\sqrt{2}} = 2\sqrt{2}.$$

Encoding all three observations gives

Direction	w_1	w_2	w_3
Horizontal	-2	0	2
Diagonal	$-2\sqrt{2}$	0	$2\sqrt{2}$

Both rows have mean zero. Squaring and averaging gives

$$V(c_h) = \frac{(-2)^2 + 0^2 + 2^2}{3} = \frac{8}{3},$$

$$V(c_d) = \frac{(-2\sqrt{2})^2 + 0^2 + (2\sqrt{2})^2}{3} = \frac{16}{3}.$$

The diagonal retains more of the original spread. Its negative and positive coordinates describe opposite sides of the same line.

3.4 Why maximum variance gives minimum error

For a unit direction c, write one observation as the sum of its reconstruction $\hat{x} = cw$, where $w = c^\top x$, and its residual $r = x - \hat{x}$. The residual is perpendicular to the chosen line. The original vector, its reconstruction, and its residual therefore form a right triangle, as in the horizontal projection diagram in Example 33.

By Pythagoras, the squared lengths satisfy

$$\|x\|_2^2 = \|\hat{x}\|_2^2 + \|r\|_2^2.$$

Since c has length 1, the squared length of $\hat{x} = cw$ is w^2. Thus

$$\|r\|_2^2 = \|x\|_2^2 - w^2 = \|x\|_2^2 - (c^\top x)^2.$$

For example, projecting $(2, 2)^\top$ horizontally gives $\hat{x} = (2, 0)^\top$ and $r = (0, 2)^\top$: the squared lengths are $8 = 4 + 4$.

3.4.1 From one point to the whole dataset

Call the average squared reconstruction error $E(c)$. Applying the same identity to every observation and averaging gives

$$\begin{aligned} E(c) &= \frac{1}{P}\sum_{p=1}^{P} \|x_p - cc^\top x_p\|_2^2 \\ &= \underbrace{\frac{1}{P}\sum_{p=1}^{P} \|x_p\|_2^2}_{\text{fixed by the data}} - \underbrace{\frac{1}{P}\sum_{p=1}^{P} \left(c^\top x_p\right)^2}_{V(c)\text{: variance retained}} . \end{aligned}$$

The first term stays the same when we change direction. Increasing the second term therefore decreases the reconstruction error by exactly the same amount.

For our three observations, the fixed term is

$$\frac{\|(-2,-2)^\top\|_2^2 + \|(0,0)^\top\|_2^2 + \|(2,2)^\top\|_2^2}{3} = \frac{8+0+8}{3} = \frac{16}{3}.$$

The two choices consequently give

$$E(c_h) = \frac{16}{3} - \frac{8}{3} = \frac{8}{3}, \qquad E(c_d) = \frac{16}{3} - \frac{16}{3} = 0.$$

These are precisely the errors computed from the reconstructed points.

Definition 3.3: First principal direction

The first principal direction is a unit vector c that maximises the variance $V(c)$ of the centred observations along it. Equivalently, it minimises the average squared error $E(c)$ of reconstruction using one direction.

Source: Spinelli, Lecture 02 (PCA), slides 8–12.

4

Optimization

Start of lecture 03 (Spinelli)

In the chapter on Data Representation, we chose weights to make a reconstruction error as small as possible. Machine learning uses the same idea: a loss function scores the error for each choice of a model's weights, and minimizing it selects the weights with the best fit according to that score. We now study how to minimize or maximize a function, beginning with one adjustable number w.

4.1 What are we trying to optimize?

Let $g(w)$ assign a numerical score to the choice w. We vary w to find the smallest or largest value of g, depending on the task.

Definition 4.1: Objective function and optimization

The *objective function* is the function whose value we want to minimize or maximize. For an objective $g(w)$, we write

$$\min_w g(w) \qquad \text{or} \qquad \max_w g(w).$$

The subscript w identifies the input we are allowed to change. When we minimize an error or a cost, the objective is also called a *loss function* or a *cost function*.

We use w_* for an input that achieves the desired optimum. Thus w_* is the chosen input, and $g(w_*)$ is its score. For minimization, $w_* = \arg\min_w g(w)$ means "choose an input at which g is smallest". For maximization, arg max has the corresponding meaning with "largest". More than one input may achieve the same best score.

4.2 Global minima and maxima

A global optimum must be best among *all* the inputs we are considering. In the following examples, w can be any real number.

Definition 4.2: Global minimum and global maximum

An input w_* is a *global minimum point* of g if

$$g(w_*) \leq g(w) \qquad \text{for every } w.$$

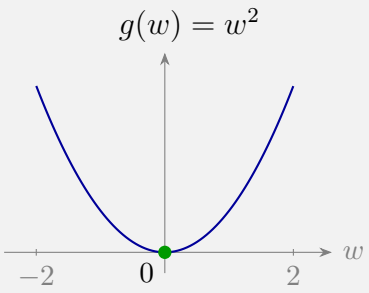
It is a *global maximum point* if

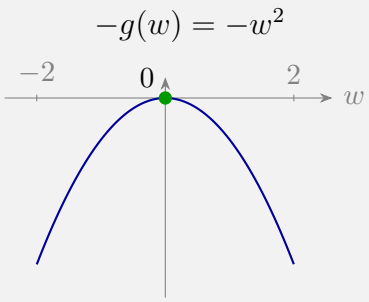
$$g(w_*) \geq g(w) \qquad \text{for every } w.$$

The values $g(w_*)$ are called the global minimum value and the global maximum value, respectively.

Example 35: The bottom of a parabola

Suppose we want to choose w to minimize $g(w) = w^2$. The horizontal axis shows the input w; the vertical axis shows the function value.





For example, $g(-2) = 4$, $g(0) = 0$, and $g(2) = 4$. Every square is nonnegative, so $g(0) = 0 \leq w^2$ for every w. Therefore $w_* = 0$ is the global minimum point, and the minimum value is 0.

If we turn the curve upside down, the function becomes $-g(w) = -w^2$. Every value is now at most 0, so the same input $w_* = 0$ is its global maximum point.

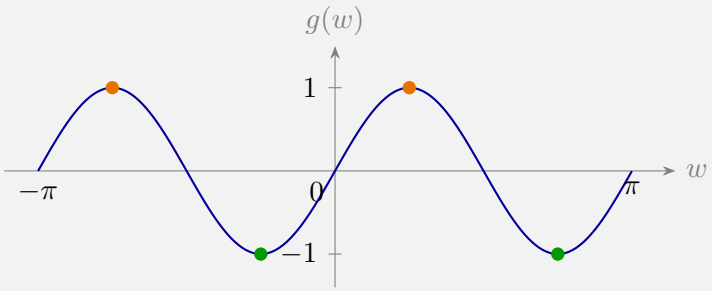
This change of sign works for any objective. If one input gives a larger value of g, it gives a smaller value of $-g$. We can therefore solve a maximization problem by minimizing the negative function:

$$\arg\max_w g(w) = \arg\min_w \big(-g(w)\big), \qquad \max_w g(w) = -\min_w \big(-g(w)\big).$$

The first equality identifies the optimizing inputs; the second relates the optimum values.

Example 36: Several inputs can be globally optimal

Consider the objective $g(w) = \sin(2w)$. Its graph repeatedly reaches the same lowest height, -1, and the same highest height, 1. Green dots mark minima; orange dots mark maxima.

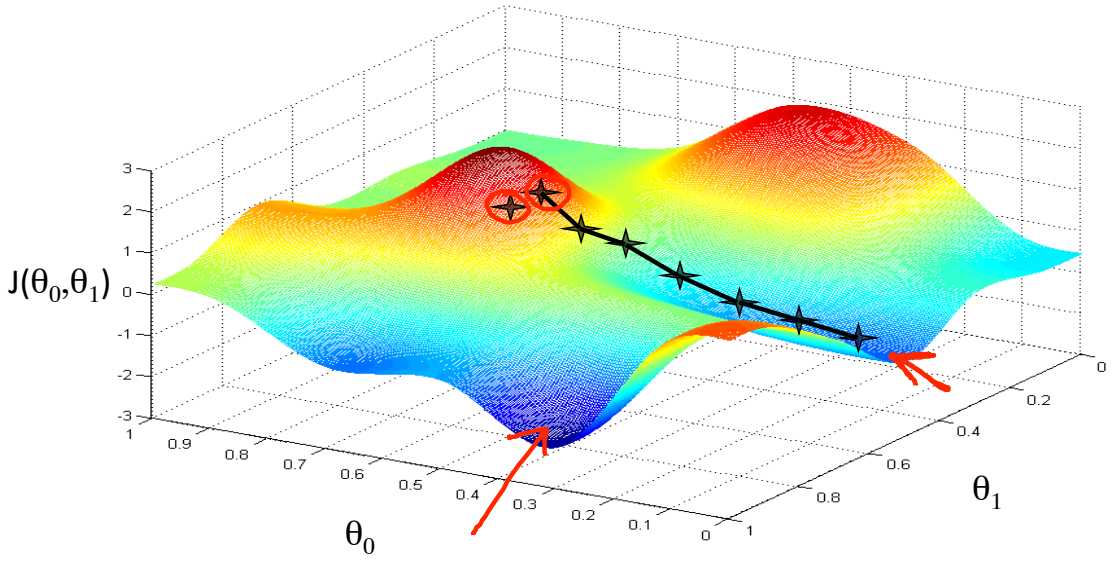


For instance, $w = -\pi/4$ and $w = 3\pi/4$ both give $g(w) = -1$. No input can produce a smaller value, so both are global minimum points. Likewise, $w = -3\pi/4$ and $w = \pi/4$ give the global maximum value 1.

The sine curve repeats, so there are infinitely many global minimum points and infinitely many global maximum points. An optimum need not be unique.

4.3 Local minima and maxima

Now imagine a curve with several valleys of different depths. The bottom of one valley is lower than the points immediately around it, even when another valley is deeper. This motivates a comparison with *nearby* inputs.



A function with several valleys. Source: Andrew Ng, Machine Learning, Lecture 2, slide 25.

Here the function has two inputs: the adjustable parameters θ_0 and θ_1 on the horizontal axes. The height $J(\theta_0, \theta_1)$ is the value we want to minimise; J plays the same role as g in our notation. The black path shows successive downhill steps into the valley on the right. The valley in front is deeper: being lower than nearby points does not necessarily mean having the lowest value overall.

Definition 4.3: Local minimum and local maximum

An input w_* is a *local minimum point* if

$$g(w_*) \leq g(w) \qquad \text{for all } w \text{ sufficiently near } w_*.$$

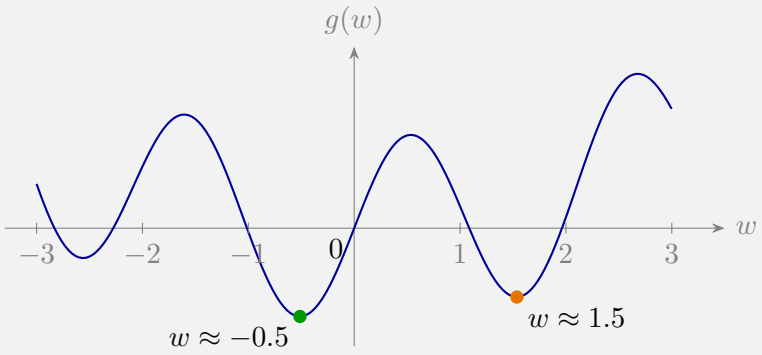
It is a *local maximum point* if the inequality is reversed. Here "sufficiently near" means that the comparison holds throughout some small interval around w_*.

Example 37: Valleys of different depths

Consider the sum of an oscillating function and a quadratic term:

$$g(w) = \sin(3w) + 0.1w^2.$$

The sine term produces peaks and valleys. The quadratic term grows as $|w|$ increases, so these valleys have different depths. The graph shows two of them with colored dots.



The green point near $w = -0.5$ is the deepest valley: its value is about -0.973. It is a global minimum point. The orange point near $w = 1.5$ is also lower than its immediate neighbors, but its value is about -0.759. It is a local minimum point with a larger objective value.

The peaks near $w \approx 0.536$ and $w \approx 2.678$ are local maximum points: their values are larger than those at nearby inputs.

Imagine searching for a minimum by making small changes that lower g. In this example, the search can stop in the orange valley. Reaching the deeper green valley requires first climbing out of the orange one. The orange point is a local minimum, but the green point has a lower value of g.

4.4 Why these are called zero-order conditions

The global and local definitions compare values of g directly. They are called *zero-order conditions* because they use the function itself. First-order conditions use first derivatives, which describe the slope of the graph. Second-order conditions use second derivatives, which describe how that slope changes. The definitions tell us what it means to be a minimum or maximum; derivatives will help us find candidate optimum points without comparing every possible input individually.

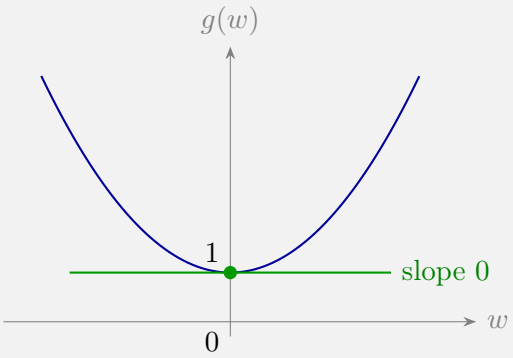
4.5 Using the derivative to find candidate optima

We now want to find possible minimum points by solving an equation. With one adjustable number w, we use the derivative $g'(w)$, also written $\frac{\mathrm{d}g}{\mathrm{d}w}$. It tells us how g changes when we change w slightly. On the graph, it is the slope of the *tangent line*: the straight line that follows the direction of the curve at the point we are examining.

If $g'(w) > 0$, moving slightly to the right increases g. If $g'(w) < 0$, the same movement decreases g. If $g'(w) = 0$, the tangent is horizontal. Let us see what happens at the bottom of a parabola.

Example 38: A horizontal tangent at the minimum

Suppose we want to minimise $g(w) = w^2 + 1$. The blue curve shows the objective, and the green line is its tangent at $w = 0$.



Differentiate w^2 to obtain $2w$; the constant 1 contributes zero. To find where the slope is zero, solve

$$g'(w) = 2w = 0 \qquad \Longrightarrow \qquad w_* = 0.$$

This point is a global minimum because $w^2 + 1 \geq 1$ for every w, and $g(0) = 1$.

Assume that the derivative exists and that we can move a little to either side of the input. At a local minimum, neither direction can lower g. A nonzero slope would let us lower it by moving in one of those directions. The slope must therefore be zero.

Definition 4.4: Stationary point and first-order condition

An input w_* is a *stationary point* if

$$g'(w_*) = 0.$$

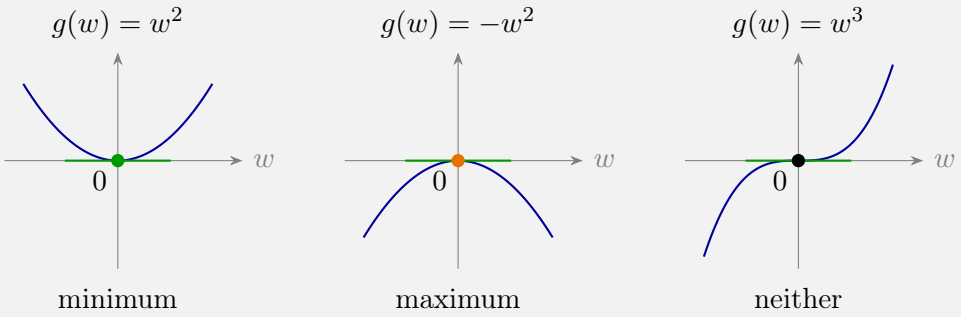
For a differentiable function, every local minimum or local maximum inside the input domain satisfies this equation. It is called the *first-order condition*. Here "inside" means that small moves to both sides are allowed.

4.6 Why a zero derivative is not enough

Solving $g'(w) = 0$ gives us points to check. A horizontal tangent can occur at a minimum, at a maximum, or at a point that is neither. The next three curves show these possibilities.

Example 39: Three stationary points with different meanings

Consider $g(w) = w^2$, $g(w) = -w^2$, and $g(w) = w^3$. Each graph has a horizontal tangent at $w = 0$.



minimum maximum neither

For $g(w) = w^2$, solving $g'(w) = 2w = 0$ gives $w = 0$. Every value of w^2 is nonnegative, so this is a minimum.

For $g(w) = -w^2$, solving $g'(w) = -2w = 0$ also gives $w = 0$. Every value of $-w^2$ is nonpositive, so this is a maximum.

For $g(w) = w^3$, the first-order equation is

$$g'(w) = 3w^2 = 0 \qquad \Longrightarrow \qquad w = 0.$$

Choose any small positive number ε. On opposite sides of 0, the function takes values

$$g(-\varepsilon) = -\varepsilon^3 < g(0) = 0 < \varepsilon^3 = g(\varepsilon).$$

We can make these inputs as close to 0 as we want. There are therefore both smaller and larger values near 0, so it is neither a minimum nor a maximum. It is a *stationary point of inflection*: the tangent is horizontal and the curve changes how it bends, but it keeps rising through the point.

The first-order condition is *necessary* at a differentiable interior minimum, but it is not *sufficient*: every such minimum has zero derivative, while some points with zero derivative are not minima. We must still examine the function around each candidate.

4.7 Several weights: a system of first-order equations

We now make the same search with several adjustable weights. Write them as the vector $w = (w_1, \dots, w_N)^\top$, where N is the number of weights. The objective $g(w)$ still returns a single score, but that score now depends on all N entries.

To check the effect of one weight, vary w_j while keeping the others fixed. The resulting rate of change is the *partial derivative* $\frac{\partial g}{\partial w_j}$. The index j identifies the weight being varied, from 1 to N.

Definition 4.5: Gradient

For a differentiable function g of N weights, the *gradient* collects its N partial derivatives in one column vector:

$$\nabla g(w) = \begin{pmatrix} \dfrac{\partial g}{\partial w_1}(w) \\ \dfrac{\partial g}{\partial w_2}(w) \\ \vdots \\ \dfrac{\partial g}{\partial w_N}(w) \end{pmatrix}.$$

The jth entry measures how g changes when we vary w_j while keeping the other weights fixed.

At a differentiable local minimum inside the input domain, every entry must be zero. Otherwise, changing one weight in the appropriate direction would lower the score.

Definition 4.6: First-order system

A stationary point w_* satisfies the N equations

$$\begin{cases} \dfrac{\partial g}{\partial w_1}(w_*) = 0, \\ \dfrac{\partial g}{\partial w_2}(w_*) = 0, \\ \qquad\qquad \vdots \\ \dfrac{\partial g}{\partial w_N}(w_*) = 0. \end{cases} \qquad \Longleftrightarrow \qquad \nabla g(w_*) = \mathbf{0}_{N\times 1}.$$

The right-hand side is a column vector of N zeros. The equations set every partial derivative to zero, not the value of g.

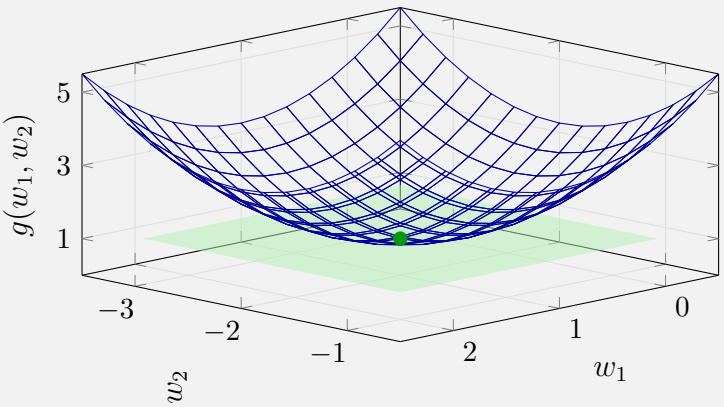
When we can solve these equations directly, we obtain an *analytic solution*, also called a *closed-form solution*. The next example shows the calculation with two weights.

Example 40: Two weights, two equations

Suppose we want to minimise

$$g(w_1, w_2) = (w_1 - 1)^2 + (w_2 + 2)^2 + 1.$$

The surface is a bowl. The green plane is its horizontal tangent at the green point $(w_1, w_2) = (1, -2)$.



Differentiate with respect to w_1, keeping w_2 fixed. The term $(w_2 + 2)^2$ is then constant and contributes zero. Reversing the roles of the weights gives the other partial derivative:

$$\frac{\partial g}{\partial w_1} = 2(w_1 - 1), \qquad \frac{\partial g}{\partial w_2} = 2(w_2 + 2).$$

Set both equal to zero and solve the two equations:

$$\begin{cases} 2(w_1 - 1) = 0 & \Longrightarrow \quad w_1 = 1, \\ 2(w_2 + 2) = 0 & \Longrightarrow \quad w_2 = -2. \end{cases}$$

Thus $w_* = (1, -2)^\top$. Both squares vanish there, so $g(w_*) = 1$. Every other value is at least 1: this candidate is a global minimum.

With several weights, a zero gradient can also identify a point that is neither a minimum nor a maximum. One direction may lead to larger values while another leads to smaller ones.

Definition 4.7: Saddle point

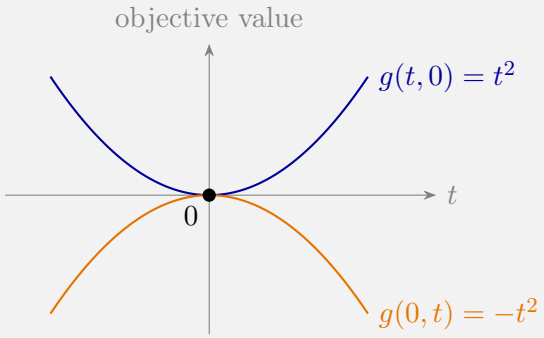
A *saddle point* of a differentiable function is a stationary point with both smaller and larger function values arbitrarily nearby. It is therefore neither a local minimum nor a local maximum.

Example 41: Zero gradient at a saddle point

Consider the two-weight objective $g(w_1, w_2) = w_1^2 - w_2^2$. Its first-order equations are

$$\frac{\partial g}{\partial w_1} = 2w_1 = 0, \qquad \frac{\partial g}{\partial w_2} = -2w_2 = 0.$$

They give the stationary point $w_* = (0, 0)^\top$, with $g(w_*) = 0$. To examine this point, let t be a small change in one weight. The graph shows the function along the two coordinate directions: $g(t, 0) = t^2$ when we change only w_1, and $g(0, t) = -t^2$ when we change only w_2.



For any nonzero t, changing only w_1 raises the value above 0, while changing only w_2 lowers it below 0. These changes can be as small as we want. The point $(0, 0)^\top$ is a saddle point, even though both partial derivatives are zero.

4.8 Approximating a function near one point

Solving $g'(w) = 0$ can become difficult when g is complicated. Another useful way to study the objective is to replace a small part of its graph with a simpler shape: its tangent line. This approximation helps us predict how a small change in a weight will affect the score.

Choose a starting input w_0. We know the value $g(w_0)$ and the slope $g'(w_0)$ there. Moving from w_0 to a nearby input w changes the input by $w - w_0$. Multiplying this change by the slope gives the predicted change in the function.

Definition 4.8: First-order Taylor approximation

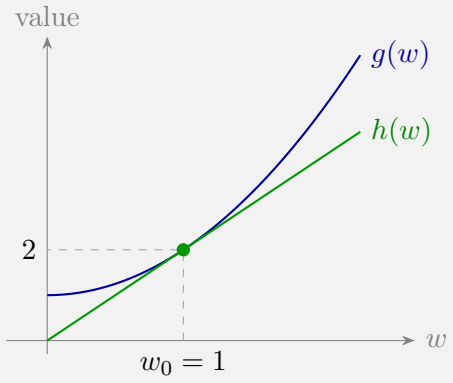
The *first-order Taylor approximation* of g at w_0 is

$$h(w) = g(w_0) + g'(w_0)(w - w_0).$$

The function h is the tangent line. It has the same value and the same slope as g at w_0. For inputs close to w_0, we use $g(w) \approx h(w)$.

Example 42: Using a tangent line to predict a nearby value

Take $g(w) = w^2 + 1$ and choose $w_0 = 1$. The green line touches the blue curve at $(1, 2)$ and follows its slope there.



At the chosen input, $g(1) = 2$ and $g'(1) = 2$. Substituting these numbers into the approximation gives

$$h(w) = 2 + 2(w - 1) = 2w.$$

For the nearby input $w = 1.1$, the line predicts $h(1.1) = 2.2$. The actual value is $g(1.1) = 1.1^2 + 1 = 2.21$, so the prediction is close.

Farther away, at $w = 2$, the line predicts 4, while the function gives 5. The tangent describes the function near the chosen point; it does not replace the whole curve.

4.8.1 From a tangent line to a tangent plane

With N weights, the starting input w_0 is a vector. The displacement $w - w_0$ contains the change in each weight. Each partial derivative predicts the effect of its corresponding change, and we add these contributions:

$$\nabla g(w_0)^\top (w - w_0) = \sum_{j=1}^{N} \frac{\partial g}{\partial w_j}(w_0)(w_j - w_{0,j}).$$

Here $w_{0,j}$ is the jth entry of the starting vector w_0. The product on the left is a dot product: it multiplies matching entries of the two vectors and sums the results.

Definition 4.9: Tangent hyperplane

For a differentiable function of several weights, the first-order Taylor approximation at w_0 is

$$h(w) = g(w_0) + \nabla g(w_0)^\top (w - w_0).$$

Its graph is the *tangent hyperplane*. It passes through the point $(w_0, g(w_0))$ and has the same partial derivatives as g there. For nearby inputs, $g(w) \approx h(w)$.

For one input, the graph is a line with two coordinates: the weight and the function value. For two inputs, it is a plane with three coordinates: two weights and the function value. In general, this flat graph has dimension N in a space with $N + 1$ coordinates.

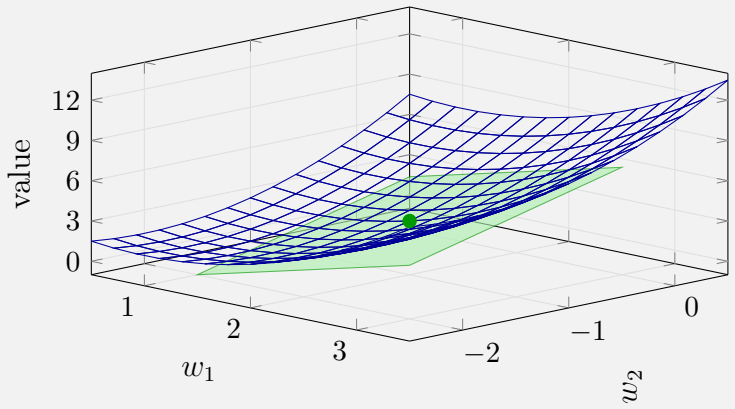
We can collect the constants and write the same model as

$$h(w) = a + b^\top w, \qquad a = g(w_0) - \nabla g(w_0)^\top w_0, \quad b = \nabla g(w_0).$$

The vector b contains the slopes, while the scalar a sets the height of the graph when $w = 0$. This is an *affine hyperplane*: the constant term allows it to be shifted away from the origin.

Example 43: A tangent plane for two weights

Use the objective $g(w_1, w_2) = (w_1 - 1)^2 + (w_2 + 2)^2 + 1$ and choose the starting weights $w_0 = (2, -1)^\top$. The green plane touches the blue surface at this point.



First compute the function value and the partial derivatives at the starting weights:

$$g(2, -1) = 1^2 + 1^2 + 1 = 3, \qquad \nabla g(2, -1) = \begin{pmatrix} 2(2-1) \\ 2(-1+2) \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

The tangent model is therefore

$$h(w_1, w_2) = 3 + 2(w_1 - 2) + 2(w_2 + 1) = 1 + 2w_1 + 2w_2.$$

In the form $h(w) = a + b^\top w$, we have $a = 1$ and $b = (2, 2)^\top$. Checking the starting point gives $h(2, -1) = 3$, exactly as required.

Changing both weights by 0.1 gives $(2.1, -0.9)^\top$. The plane predicts 3.4, while the actual function value is $1.1^2 + 1.1^2 + 1 = 3.42$.

4.9 Choosing the direction of fastest change

The tangent model helps us choose how to change several weights together. With one weight, we can move left or right; the sign of the slope determines which way increases the function. With two or more weights, there are infinitely many directions to consider.

Let $d = (d_1, \ldots, d_N)^\top$ describe a direction. To compare directions fairly, give them the same Euclidean length:

$$\|d\| = \sqrt{d_1^2 + \cdots + d_N^2} = 1.$$

Such a vector is called a *unit direction*. If $\alpha > 0$ is the distance we choose to move, the new weights are $w_0 + \alpha d$. All the directions being compared then produce steps of length α.

For the affine model $h(w) = a + b^\top w$, this move gives

$$h(w_0 + \alpha d) = a + b^\top w_0 + \alpha b^\top d.$$

The first two terms are fixed. The predicted change is $\alpha b^\top d$, so the direction with the largest dot product $b^\top d$ gives the largest increase; the smallest gives the largest decrease.

Assume $b \neq 0$ and let θ be the angle between b and d. The dot product satisfies

$$b^\top d = \|b\| \, \|d\| \cos\theta = \|b\| \cos\theta.$$

The length $\|b\|$ is fixed. The largest value occurs when $\cos\theta = 1$, so d points along b. The smallest occurs when $\cos\theta = -1$, so d points opposite to b.

Definition 4.10: Steepest ascent and steepest descent

At an input w_0 where $\nabla g(w_0) \neq 0$, the Euclidean unit directions of fastest local increase and decrease are

$$d_{\text{ascent}} = \frac{\nabla g(w_0)}{\|\nabla g(w_0)\|}, \qquad d_{\text{descent}} = -\frac{\nabla g(w_0)}{\|\nabla g(w_0)\|}.$$

The largest rate of increase per unit distance is $\|\nabla g(w_0)\|$. The gradient itself points in the ascent direction, and its negative points in the descent direction. Dividing by the length makes these vectors unit directions.

The tangent model has the same slopes as the original function at w_0, so these are also the fastest *local* directions for g. A sufficiently small step opposite to a nonzero gradient lowers g; a large step can increase it. Choosing a direction and choosing how far to move are therefore separate parts of the search.

At a stationary point, the gradient is zero and the tangent model is horizontal. It gives no preferred direction: we must examine more than this first-order approximation to understand the function there.

Example 44: Equal step lengths, different changes

For $g(w_1, w_2) = (w_1 - 1)^2 + (w_2 + 2)^2 + 1$ at $w_0 = (2, -1)^\top$, the tangent model is $h(w) = 1 + 2w_1 + 2w_2$ and $b = (2, 2)^\top$. Compare the following unit directions. The dot product gives the change in h per unit distance.

Unit direction d	$b^\top d$	Effect on h
$(1, 0)^\top$	2	increase
$(0, 1)^\top$	2	increase
$(1, -1)^\top/\sqrt{2}$	0	no change
$(1, 1)^\top/\sqrt{2}$	$2\sqrt{2}$	fastest increase
$(-1, -1)^\top/\sqrt{2}$	$-2\sqrt{2}$	fastest decrease

The gradient length is $\sqrt{2^2 + 2^2} = 2\sqrt{2}$. Dividing the negative gradient by this length gives

$$d_{\text{descent}} = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ -1 \end{pmatrix}.$$

A step of length α in this direction gives the new weights

$$w_0 + \alpha d_{\text{descent}} = \begin{pmatrix} 2 - \alpha/\sqrt{2} \\ -1 - \alpha/\sqrt{2} \end{pmatrix}.$$

The tangent model predicts the value $3 - 2\sqrt{2}\alpha$. Substituting the new weights into the actual objective gives

$$g(w_0 + \alpha d_{\text{descent}}) = 2\left(1 - \frac{\alpha}{\sqrt{2}}\right)^2 + 1 = 3 - 2\sqrt{2}\alpha + \alpha^2.$$

For $\alpha = 0.1$, the predicted value is about 2.717, and the actual value is about 2.727. Both are below the starting value 3. The extra α^2 term shows the part that the tangent model does not capture.

4.10 Level curves and the gradient

We can also view a two-weight objective from above. Instead of drawing its height, we connect inputs with the same function value, as a map connects places at the same altitude.

Definition 4.11: Level set and level curve

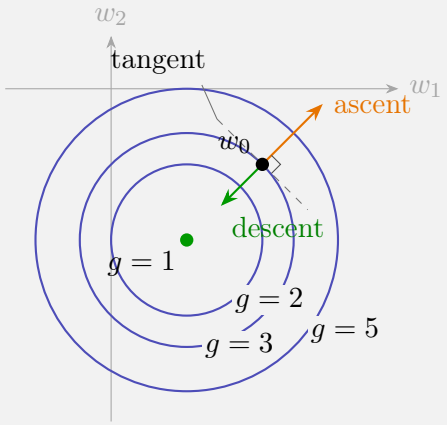
For a fixed score c, the *level set* of g is

$$\{w : g(w) = c\}.$$

With two inputs, these sets are usually drawn as *level curves*. Every point on one curve has the same score c. Different curves correspond to different scores.

Example 45: Reading the gradient on a contour map

For $g(w_1, w_2) = (w_1 - 1)^2 + (w_2 + 2)^2 + 1$, the level curves are circles centred at the minimum $(1, -2)^\top$, where $g = 1$. The black point $w_0 = (2, -1)^\top$ has score 3.



The gradient $(2, 2)^\top$ points outward, towards larger scores; its negative points inward, towards smaller scores. The dashed tangent direction $t = (1, -1)^\top$ is perpendicular to it:

$$\nabla g(w_0)^\top t = 2 \cdot 1 + 2 \cdot (-1) = 0.$$

Moving along the level curve keeps the score constant, so the first-order change in its tangent direction is zero.

Where the gradient is nonzero, it is perpendicular to the level curve. In higher dimensions, it is perpendicular to the surface of constant function value.

Sources: Spinelli, Lecture 03 (Optimization), slides 2–29; Watt, Borhani, and Katsaggelos, Machine Learning Refined (2016), Sections 1.4, 2.1.1–2.1.2, and 2.2.1.

5

Shallow Neural Networks

Start of lecture 05 (Spinelli) — 2 November 2026

5.1 Choosing features by hand

In traditional machine learning (e.g., SVMs, Logistic Regression), the model's performance depends entirely on the quality of the input features.

Imagine that we want to recognise spam emails. We have the text of each message, but a model such as logistic regression needs a numerical description. What should those numbers tell it? One approach, as we have seen before, is to choose features that we think will help.

Example 46: Turning an email into three numbers

Consider the message

> Buy Viagra now for cheap!!!

We decide to describe it using three features:

Feature chosen by us	Value
Contains the word "viagra"	1
Number of exclamation marks	3
Contains the word "cheap"	1

The message is now represented by

$$x = \begin{pmatrix} 1 \\ 3 \\ 1 \end{pmatrix}.$$

Each entry records one measurement. The first and third entries are indicators: 1 means that the word is present, and 0 would mean that it is absent.

Given many emails labelled spam or non-spam, the classifier learns how to use these numbers to make its decision. We chose what to measure; training learns how those measurements affect the prediction.

Definition 5.1: Feature engineering

Feature engineering is the process of designing or choosing measurements and transformations that turn the original data into features for a model.

This approach relies on us knowing what information is useful. If our chosen measurements leave out an important pattern, the classifier cannot use that pattern from the supplied features.

Example 47: Different messages, the same description

Now consider a suspicious message:

> You have won a prize. Enter your card details now.

It contains neither of our chosen words and no exclamation marks. The three features chosen in Example 46 give

$$x = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

An ordinary message such as "See you at class tomorrow." gives exactly the same vector. With only these features, the classifier receives the same input for both messages and cannot distinguish them. We left out the information that would let it do so.

5.2 Learning the representation

Could the model learn a useful description of the message as well as learning how to classify it? This is the idea behind *representation learning*.

The text still needs an initial numerical encoding. The model then learns transformations of that input, producing internal features that help it perform its task.

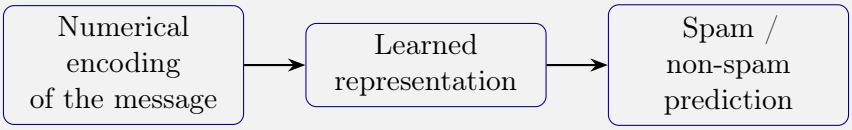
Definition 5.2: Representation learning

Representation Learning is a set of techniques that allows a model to automatically learn the most useful features (or "representations") directly from the raw data.

Example 48: Learning features that help recognise spam

Return to the email task. We supply messages and their spam/non-spam labels. During training, a neural network can learn transformations that exploit combinations of words and other patterns associated with these labels.

We do not need to enumerate every suspicious expression in advance. The learned internal description helps the model make its final classification:



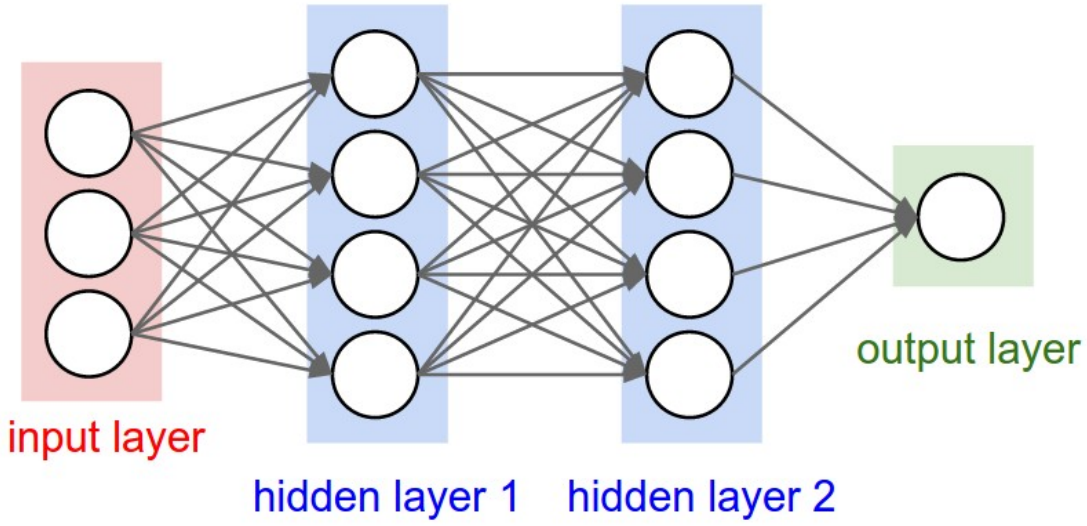
Both the transformations producing useful features and the rule using those features for classification can be learned during training.

5.3 Learning through several layers

A useful representation can be built through several transformations. One layer produces a collection of numbers; the next layer uses those numbers as its input and transforms them again. This lets later stages combine patterns detected by earlier ones.

Definition 5.3: Deep learning

Deep learning uses neural networks with multiple successive learned layers to build a hierarchy of representations. These networks are called **deep neural networks (DNNs)**. Each layer transforms the output of the preceding layer, allowing later representations to combine simpler patterns into more complex ones.

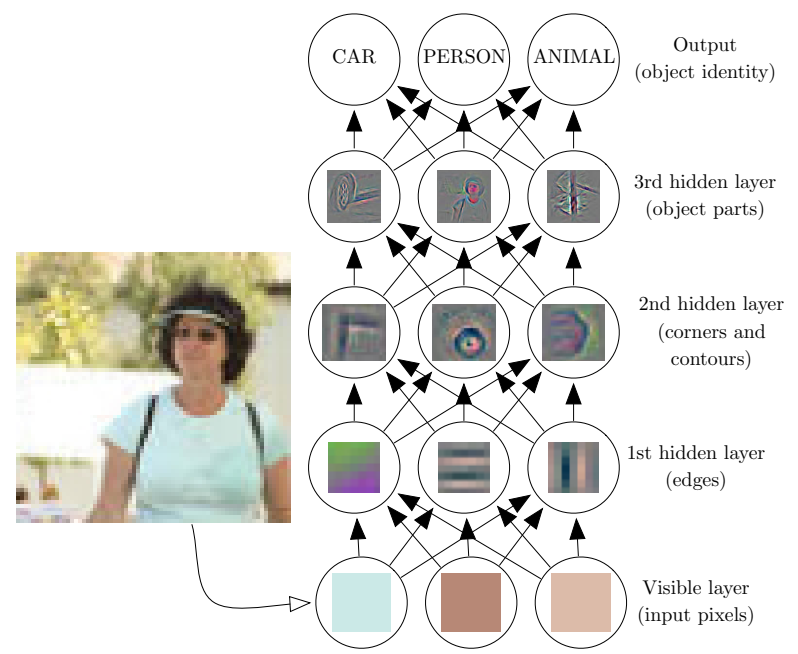


Input values pass through two hidden layers before reaching the output layer. Each layer uses the values produced by the previous one.

Example 49: Recognising a face, layer by layer

Imagine a deep neural network learning to recognise a face. Its input consists of numbers describing the image's pixels. We can understand how increasingly complex representations are built by following four stages:

1. **Layer 1 (Low-level): simple visual features.** Starting from the raw pixels, this layer learns to detect edges, corners, and colours. These give a first description of local patterns in the image.
2. **Layer 2 (Mid-level): shapes and textures.** This layer uses the features from Layer 1. It combines edges into simple shapes and textures, such as an oval, a line, or a patch of skin.
3. **Layer 3 (High-level): parts of objects.** This layer combines those shapes into more complex patterns, such as an eye, a nose, or a mouth.
4. **Final layer (Abstract): recognising the face.** The final layer combines information about these parts to recognise the overall concept of a face.



Each stage builds on the previous one: pixels give rise to simple features, simple features combine into shapes, and those shapes contribute to recognising parts and then the whole face.

These stages describe how information flows through the network. During training, the parameters in different layers can be adjusted together.

5.4 A unifying view of models we already know

We may want a model to predict a number, decide between two classes, or choose among several classes. These tasks lead to linear regression, binary classification, and multi-class classification, respectively. All three can start with the same calculation: combine the input features using weights, then transform the result into the output we need.

For one observation, suppose our three features are

$$x = \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix}.$$

We multiply each feature by a weight and add the results. Each weight controls its feature's contribution; a zero weight leaves that feature out of the calculation.

Definition 5.4: Activation function

An **activation function** transforms the result of a weighted combination of inputs. Here, we choose this function according to the output we need: a numerical prediction, a probability for one class, or probabilities for several classes.

5.4.1 Linear regression: predicting a number

Example 50

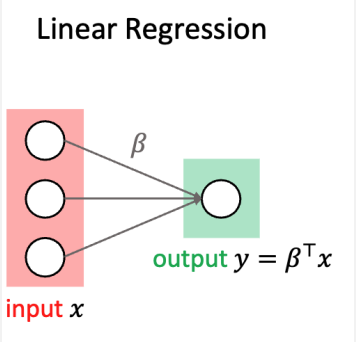
Use the input $x = (2, 1, 3)^\top$ above and, for this example, choose weights $\beta = (1, -1, 0)^\top$. Their weighted combination is

$$\beta^\top x = 1 \cdot 2 - 1 \cdot 1 + 0 \cdot 3 = 1.$$

In linear regression, this number is already our prediction:

$$y = \beta^\top x = 1.$$

We are predicting a numerical quantity, so we leave the result as it is. We can describe this using the **identity activation**, $g(z) = z$: it simply returns the number it receives.



5.4.2 Binary classification: turning a score into a probability

Example 51

Keep the same input and weights as in Example 50, but now imagine that we must decide whether an email is spam. The weighted combination still gives 1. This is a **score**; it is not yet a probability. With other inputs or weights, the score could be negative or greater than one.

We pass it through the **sigmoid function**,

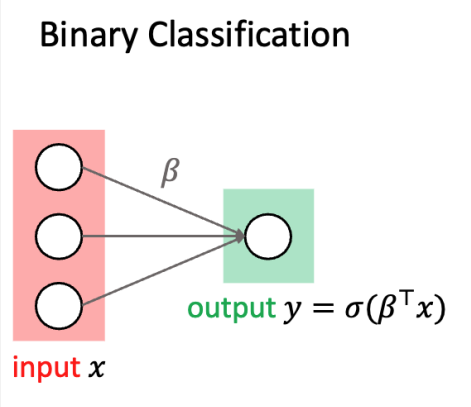
$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

which maps any real score to a number between zero and one:

$$y = \sigma(\beta^\top x) = \sigma(1) \approx 0.73.$$

If class 1 means spam, the model assigns a spam probability of about 73%. The calculation now follows this path:

$$\text{three features} \longrightarrow \text{one score} \longrightarrow \text{one probability}.$$



5.4.3 Multi-class classification: one score for each class

Example 52

Suppose we use three image features to classify an image as a cat, a dog, or a bird. We need a score for each class. Each class therefore has its own weights, which we collect as rows of a matrix W.

For a simple numerical example, keep $x = (2, 1, 3)^\top$ and choose

$$W = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad Wx = \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix} \begin{matrix} \text{cat} \\ \text{dog} \\ \text{bird.} \end{matrix}$$

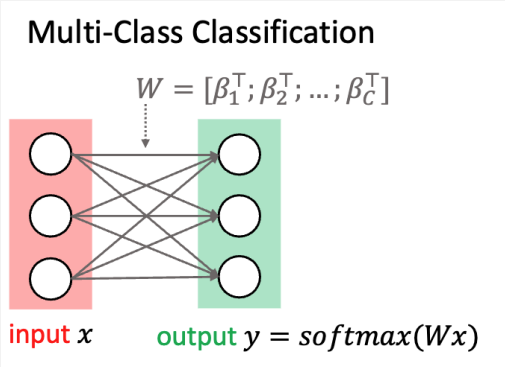
These are three scores, not three probabilities. The **softmax function** converts the scores into probabilities that sum to one. For scores $z_1, \ldots, z_C$, its cth output is

$$\text{softmax}(z)_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}.$$

In our example, this gives

$$y = \text{softmax}(Wx) = \frac{1}{e^2 + e^1 + e^3} \begin{pmatrix} e^2 \\ e^1 \\ e^3 \end{pmatrix} \approx \begin{pmatrix} 0.245 \\ 0.090 \\ 0.665 \end{pmatrix}.$$

The highest probability belongs to bird, so that is the predicted class. The largest score also receives the largest probability.



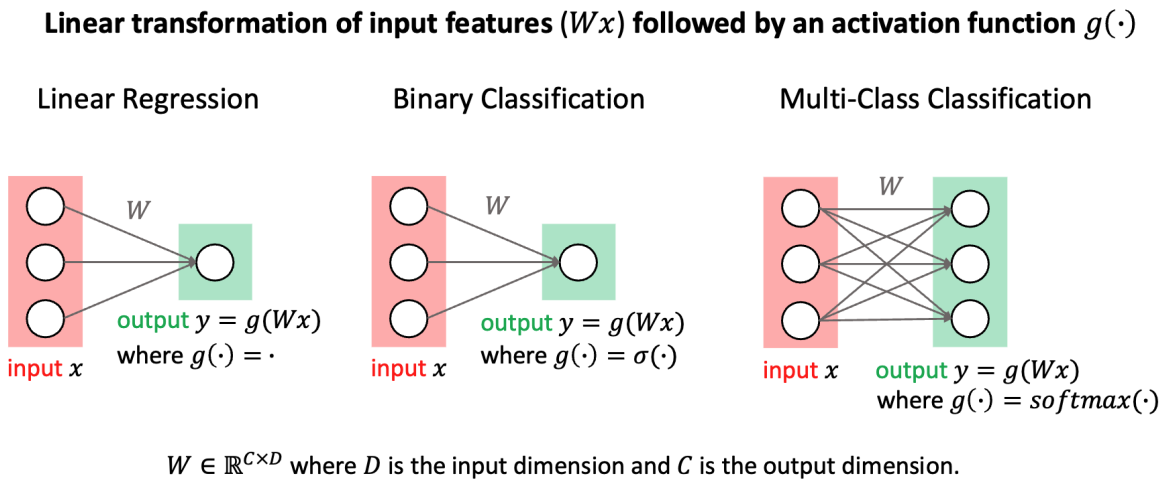
5.4.4 The same two steps in one formula

We can write all three calculations with a single formula:

$$\boxed{y = g(Wx).}$$

First, Wx combines the features to produce the scores. Then g transforms those scores into the required output. For linear regression, g is the identity; for binary classification, it is the sigmoid; for multi-class classification, it is the softmax.

The notation W also covers the single-output models: in those cases, W is just the row vector previously written as $\beta^\top$.



Definition 5.5: Input and output dimensions

With D input features and C output scores, the weight matrix has C rows and D columns:

$$x \in \mathbb{R}^D, \qquad W \in \mathbb{R}^{C \times D}, \qquad Wx \in \mathbb{R}^C.$$

Each row of W contains the weights used to calculate one output score.

5.5 Why we need non-linear activations

Adding layers gives us more successive calculations. But do more calculations necessarily let us represent a more complex function? Let us first build a network in which every layer only multiplies its input by a weight.

Example 53

Take one input number x. The first layer multiplies it by 2, the second by 3, and the third by 4:

$$x \xrightarrow{\times 2} 2x \xrightarrow{\times 3} 6x \xrightarrow{\times 4} 24x.$$

We could obtain exactly the same output with a single layer that multiplies x by 24. This works for every input, not just for one particular numerical example. Three layers have changed how we calculate the answer, but the function is still $y = 24x$.

With several features, matrices replace the individual weights.

Example 54

For three matrix layers, the calculation becomes

$$y = W_3\big(W_2(W_1x)\big) = (W_3W_2W_1)x = W_{\text{stacked}}x.$$

Here $W_{\text{stacked}} = W_3W_2W_1$ is a single weight matrix that combines the effects of all three layers. Multiplying the input by this matrix gives the same output as passing it through the three layers in sequence.

The underlying algebraic fact is the same: a composition of linear maps is still linear. Let us review what it means for a map to be linear.

Definition 5.6: Linear map

A map T is **linear** if it preserves linear combinations: for any input vectors u, v and scalars a, b,

$$T(au + bv) = aT(u) + bT(v).$$

Thus, combining inputs before applying T gives the same result as applying T to each input and then combining the outputs. Multiplication by a fixed matrix, $T(x) = Wx$, is a linear map.

5.5.1 The algebra behind a stack of linear layers

Proposition 5.1: Composition of linear maps

Let $T : \mathbb{R}^D \to \mathbb{R}^H$ and $S : \mathbb{R}^H \to \mathbb{R}^C$ be linear maps. Their composition $(S \circ T)(x) = S(T(x))$ is linear. Consequently, any finite composition of linear maps is linear.

Proof. Take $u, v \in \mathbb{R}^D$ and scalars a, b. First use the linearity of T, then the linearity of S:

$$\begin{aligned}(S \circ T)(au + bv) &= S\big(T(au + bv)\big) \\ &= S\big(aT(u) + bT(v)\big) \\ &= aS(T(u)) + bS(T(v)) \\ &= a(S \circ T)(u) + b(S \circ T)(v).\end{aligned}$$

This is exactly the condition for linearity. For more maps, apply the same argument repeatedly, combining two maps at a time. □

5.5.2 What changes when we insert a non-linearity?

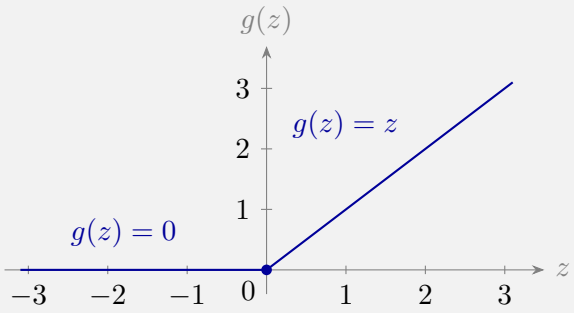
We have just seen that composing linear transformations alone is not enough to build a non-linear model. We therefore need to introduce non-linearity somewhere in the calculation. How can we do this? One way is to choose the ReLU activation as an activation function between layers.

Definition 5.7: Rectified Linear Unit (ReLU)

The **ReLU activation** is

$$g(z) = \max(0, z).$$

It returns z when $z > 0$ and zero otherwise. For a vector, we apply it separately to each entry. Although its name contains "linear", ReLU is not a linear map: its rule changes at zero (it is easy to see that $g(-1) + g(1) \neq g(-1 + 1)$).



Example 55

Use two scalar layers with weights 2 and 3, and insert ReLU between them:

$$y = 3g(2x) = 3\max(0, 2x).$$

For example, let's follow two inputs through the same network:

Input x	First layer $2x$	ReLU $g(2x)$	Output $3g(2x)$
1	2	2	6
-1	-2	0	0

Is there a single weight a such that $ax = 3g(2x)$ for every input x?

The input $x = 1$ would force $a = 6$. But then $x = -1$ would give LHS $= -6$, whereas the RHS would give 0. No single weight works for both inputs.

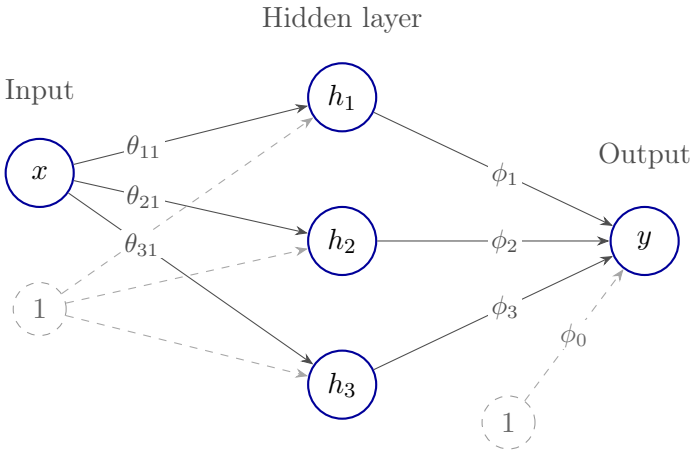
With non-linear activations between layers, the calculation generally cannot be collapsed into one linear map. Successive layers can build more complex transformations of the input.

5.6 An example shallow network

We can now write a complete network with one input x, one output y and a single hidden layer of three neurons. Each hidden neuron computes its own line of x and passes it through a generic activation function called a; the output adds up the three results with weights and a constant:

$$\begin{aligned} y &= f(x, \boldsymbol{\phi}) \\ &= \phi_0 + \phi_1 \underbrace{a(\theta_{10} + \theta_{11}x)}_{h_1} + \phi_2 \underbrace{a(\theta_{20} + \theta_{21}x)}_{h_2} + \phi_3 \underbrace{a(\theta_{30} + \theta_{31}x)}_{h_3}. \end{aligned}$$

A common choice for a is the ReLU of Definition 5.7, $a(z) = \max(0, z)$.



The dashed node 1 carries the constants: multiplied by θ_{k0} it enters each hidden neuron, multiplied by ϕ_0 it enters the output.

Unlike $y = g(Wx)$, each layer now has a constant term: $\theta_{10}, \theta_{20}, \theta_{30}$ in the hidden layer and ϕ_0 in the output. Without them every line would pass through the origin (so, for example, if a is the ReLU every hidden neuron would switch on and off at the same point $x = 0$).

The network of Example 55 is the special case with ReLU as a, one hidden neuron, $\theta_{11} = 2$, $\phi_1 = 3$ and all constants equal to zero.

Definition 5.8: Parameters of a network

The **parameters** of a network are all the numbers that training can adjust. The network above has 10:

$$\boldsymbol{\phi} = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}.$$

Each of the three hidden neurons has 2 parameters, the output has 3 weights and 1 constant: $3 \cdot 2 + 3 + 1 = 10$.

The formula therefore describes a **family of functions**, and each choice of the 10 parameters picks one particular function.

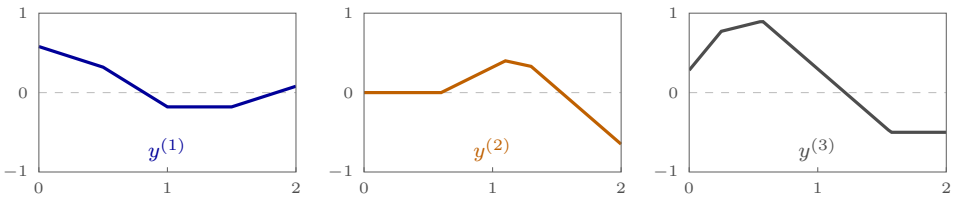
Definition 5.9: Inference

Inference means evaluating the network with its parameters fixed, to obtain the output y for a new input x.

Definition 5.10: Training

Training means choosing the parameters from data. Given a dataset $\{(x_i, t_i)\}_{i=1}^{n}$ of inputs and correct targets, we choose a loss function $L(\boldsymbol{\phi})$ that measures the prediction errors, and change the parameters to make it as small as possible.

Now let us look at some outputs of our network. We will plot three of them, $y^{(1)}$, $y^{(2)}$ and $y^{(3)}$. The network is the same in all three cases: one input, three hidden neurons with ReLU as a, one output. What changes is the value of the ten parameters in $\boldsymbol{\phi}$. The specific values are not important.



The three outputs look quite different, but they have something in common: each one is made of straight segments joined at corners. Why does a network with three ReLU neurons produce this kind of shape? Let us follow the calculation step by step.

5.6.1 Hidden units

We can break the formula into two parts.

Definition 5.11: Hidden unit

The k-th **hidden unit** is the value computed by hidden neuron k:

$$h_k = a(\theta_{k0} + \theta_{k1}x), \qquad k = 1, 2, 3.$$

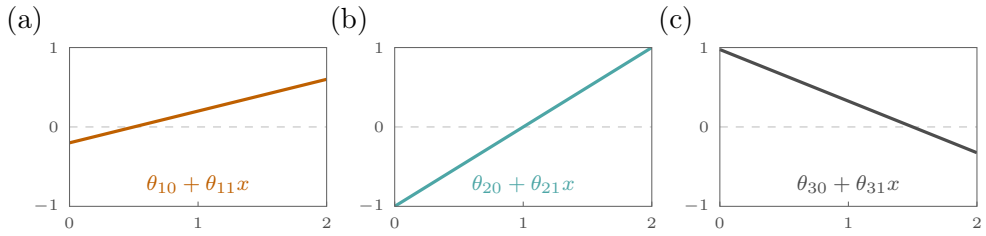
The output then combines the hidden units:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3.$$

5.6.2 Building the output step by step

For example, let's build $y^{(1)}$, the first output plotted above, for every input x between 0 and 2.

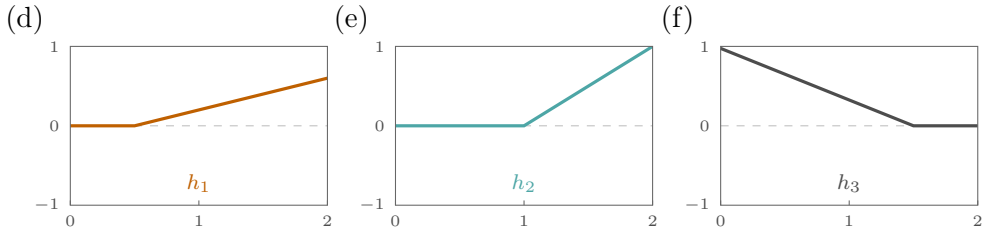
Step 1: compute three linear functions. Each hidden neuron computes a line $\theta_{k0} + \theta_{k1}x$.



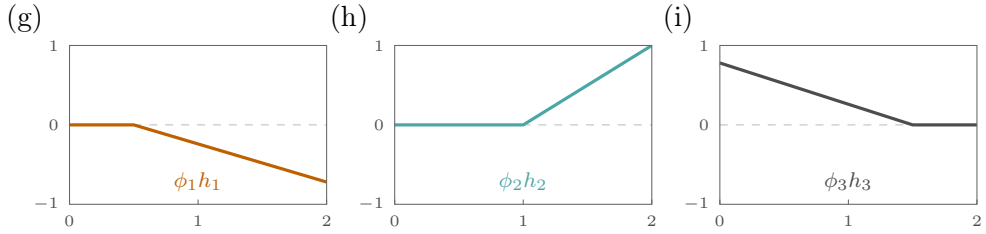
Step 2: pass each line through ReLU. This gives the hidden units

$$h_1 = a(\theta_{10} + \theta_{11}x), \qquad h_2 = a(\theta_{20} + \theta_{21}x), \qquad h_3 = a(\theta_{30} + \theta_{31}x).$$

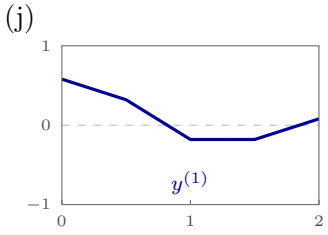
Each hidden unit has one corner, where its line crosses zero.



Step 3: weight the hidden units. Multiplying h_k by ϕ_k stretches or shrinks it, and a negative ϕ_k flips it upside down (here $\phi_1 < 0$). The flat part stays at zero.



Step 4: add the weighted hidden units and the constant ϕ_0. This gives the output y.

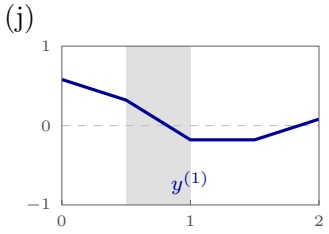


The output is a **piecewise linear function**: straight segments joined at corners, or *joints*. Each joint appears where one hidden unit switches on or off, here at $x = 0.5$, 1 and 1.5. A network with three ReLU units therefore has at most three joints.

Definition 5.12: Activation pattern

For a given input x, a hidden unit h_k with ReLU activation is **active** if $h_k > 0$ and **inactive** if $h_k = 0$. The **activation pattern** is the list of which hidden units are active.

Between two consecutive joints the activation pattern does not change. The inactive units contribute zero, and each active unit contributes a line, so y is a line whose slope is the sum of $\phi_k \theta_{k1}$ over the active units. In the shaded region below, h_1 and h_3 are active and h_2 is inactive:



For each region between two joints:

Region	Active units
$0 \leq x < 0.5$	h_3
$0.5 \leq x < 1$	h_1, h_3
$1 \leq x < 1.5$	h_1, h_2, h_3
$1.5 \leq x \leq 2$	h_1, h_2

5.7 More hidden units

So far we have worked with a network with only three hidden units. How can we generalise it to D hidden units, and what does the construction of y look like then?

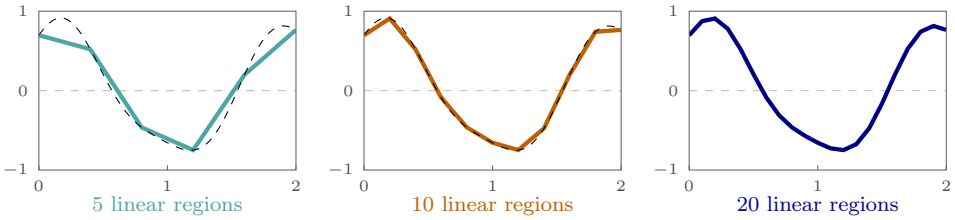
The four steps stay the same; only the number of terms changes:

1. compute D linear functions $\theta_{d0} + \theta_{d1}x$, for $d = 1, \ldots, D$;
2. pass each one through the activation, obtaining the hidden units $h_d = a(\theta_{d0} + \theta_{d1}x)$;
3. weight each hidden unit, obtaining $\phi_d h_d$;
4. add the weighted hidden units and the constant ϕ_0:

$$y = \phi_0 + \sum_{d=1}^{D} \phi_d h_d = \phi_0 + \sum_{d=1}^{D} \phi_d \, a(\theta_{d0} + \theta_{d1}x).$$

For $D = 3$ this is exactly the network of Section 5.6. Each hidden unit has 2 parameters and the output has D weights and 1 constant, so the network has $3D + 1$ parameters (10 for $D = 3$). From now on, D denotes the number of hidden units.

With ReLU, each hidden unit adds at most one joint, so the output has at most D joints and at most $D + 1$ linear regions. More hidden units therefore allow more segments, and a function with more segments can follow a curve more closely. In the plots below the dashed curve is a target function, and the solid line is a piecewise linear function with 5, 10 and 20 linear regions:



With enough hidden units, the approximation can be made as accurate as we want. This is the content of the following theorem.

Theorem 5.1: Universal approximation theorem

With enough hidden units, a shallow neural network can describe any continuous function on a compact subset of $\mathbb{R}^{D_i}$ to arbitrary precision.

Here D_i is the number of inputs, and a *compact* subset is one that is closed and bounded, such as the interval $[0, 2]$ used in the plots. The theorem says that a suitable network *exists*; it does not say how many hidden units are needed, or how to find the parameters.

5.8 More outputs

So far our network has had a single output y. Sometimes, however, we want several results from the same input, for example two different predictions. Instead of building two separate networks, we use a single one: the hidden units are computed once, and each output combines them with its own weights.

Take one input, four hidden units and two outputs. The hidden units are computed as before:

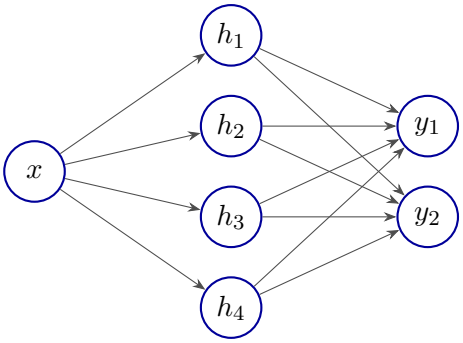
$$h_d = a(\theta_{d0} + \theta_{d1}x), \qquad d = 1, \ldots, 4.$$

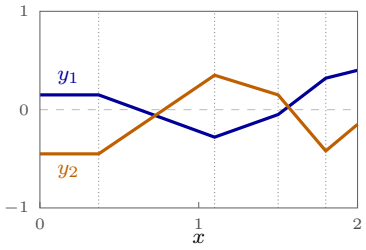
Each output then takes its own weighted sum, with its own weights and its own constant:

$$\begin{aligned} y_1 &= \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4, \\ y_2 &= \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4. \end{aligned}$$

- fixed

The output weights now have two indices: ϕ_{jd} is the weight from h_d to y_j. In the diagram, each hidden unit therefore has two outgoing arrows, one to y_1 and one to y_2.





While y_1 and y_2 are different functions, they change slope at the same points (the dotted lines). The reason is that a joint appears where a hidden unit switches on or off. The hidden units are the same for both outputs, so the points where they switch are the same too. The weights ϕ_{jd} only decide *how much* and *in which direction* each output bends at those points, not *where*.

Developing intuition

Why would we want to calculate more outputs y?

Suppose the input x is the hour of the day, from 0 to 24, and we want to predict two things at that hour: the temperature y_1 and the humidity y_2.

We could build two separate networks, one for each output. But both quantities depend on the same moments of the day: sunrise, the hottest hours in the early afternoon, sunset. With a single network, the hidden units can learn these moments once, and each output then uses them in its own way. When the sun rises, for example, the temperature goes up while the humidity usually goes down.

This is also why the two outputs change slope at the same times. A joint is an hour at which a hidden unit switches on or off, for example at sunrise or at sunset, and these hours are the same for both outputs.

5.9 How much can a network represent?

Training can only choose among the functions that the network can represent. Capacity concerns how complex those functions can be.

Definition 5.13: Model capacity, depth, and width

Model capacity describes the range and complexity of functions a model can represent. For a feed-forward network, two architectural choices that affect capacity are:

- **Depth**: the number of hidden layers.
- **Width**: the number of neurons in a hidden layer. Different hidden layers may have different widths.

A hidden layer is an intermediate layer between the input and output. Each of its neurons computes one value, so its width determines how many values it passes to the next layer.

Example 56

Compare one hidden layer of 100 neurons with five hidden layers of 20 neurons each. The first network is wider: it produces 100 hidden values at once. The second is deeper: it performs five successive hidden transformations, each producing 20 values. Both have 100 hidden neurons in total, but their arrangement changes the functions they can represent.

Increasing width or depth can increase capacity, depending on the architecture and activations. By Proposition 5.1, adding only linear layers still gives a linear map.

Greater capacity does not automatically improve predictions on new data: a flexible network may also fit accidental details of its training examples.